

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ
MINH**

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



KHÓA LUẬN TỐT NGHIỆP

**THIẾT KẾ VÀ TỐI ƯU HÓA QUANTIZED
AUTOENCODER TRÊN FPGA ARTIX-7**

NGÀNH: HỆ THỐNG NHÚNG VÀ IoT

Sinh viên: **TRỊNH PHI HÀO**

MSSV: 22139020

LÊ DUY BẢO

MSSV: 22139003

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

KHÓA LUẬN TỐT NGHIỆP

**THIẾT KẾ VÀ TỐI ƯU HÓA QUANTIZED
AUTOENCODER TRÊN FPGA ARTIX-7**

NGÀNH: HỆ THỐNG NHÚNG VÀ IoT

Sinh viên: **TRỊNH PHI HÀO**

MSSV: 22139020

LÊ DUY BẢO

MSSV: 22139003

Giảng viên hướng dẫn: TS. Nguyễn Mạnh Hùng

TP. HỒ CHÍ MINH – 06/2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ TỐI ƯU HÓA QUANTIZED AUTOENCODER TRÊN FPGA ARTIX-7.....

Sinh viên: + TRỊNH PHI HÀO.....

MSSV: 22139020

+ LÊ DUY BẢO.....

MSSV: 22139003

Hướng dẫn: Nguyễn Mạnh Hùng.....

Nhận xét bao gồm các nội dung sau đây:

- Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:
Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.
Đề tài mang tính cấp thiết về Edge AI/TinyML. Mục tiêu rõ ràng, phương pháp tối ưu hóa QAT 8-bit trên FPGA có tính sáng tạo
- Phương pháp thực hiện/ phân tích/ thiết kế:
Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.
Kỹ thuật tối ưu RTL và công cụ hls4ml được vận dụng, hiệu quả
- Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:
Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.
Phân tích, đối chiếu phần mềm - phần cứng chặt chẽ, định lượng rõ ràng
- Kết luận và đề xuất:
Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.
Kết luận phù hợp với mục tiêu đề tài, chứng minh được tính khả thi của việc triển khai mô hình học sâu trên thiết bị biên
- Hình thức trình bày và bố cục báo cáo:
Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.
Bố cục khoa học, trình bày chín chu, đúng quy định
- Kỹ năng chuyên nghiệp và tính sáng tạo:
Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.
Nắm vững công cụ (Vitis HLS, QKeras), kỹ năng nghiên cứu độc lập và làm việc nhóm tốt
- Tài liệu trích dẫn
Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.
Đầy đủ, trung thực, đúng chuẩn
- Đánh giá về sự trùng lặp của đề tài
Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.
Không trùng lặp, là công trình độc lập.
- Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*:
Cần mở rộng kiến trúc sang tập dữ liệu phức tạp hơn và thử nghiệm trên phần cứng thực tế.
- Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên
Nghiêm túc, chủ động, luôn bám sát tiến độ hướng dẫn.
Đề nghị của giảng viên hướng dẫn
Ghi rõ: "Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"
Báo cáo đạt yêu cầu, sinh viên được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 24 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)

* Giảng viên hướng dẫn có thể ghi tiếp vào mặt sau

LỜI CAM ĐOAN

Nhóm thực hiện gồm Trịnh Phi Hào và Lê Duy Bảo, sinh viên ngành Hệ thống nhúng và IoT, khoa Điện - Điện tử, Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, xin cam đoan đề tài "Thiết kế và tối ưu hóa Quantized Autoencoder trên FPGA Artix-7" do chính nhóm tự thực hiện dưới sự hướng dẫn của TS. Nguyễn Mạnh Hùng.

Toàn bộ phần thiết kế mô hình, mã nguồn, quá trình tổng hợp phần cứng và các số liệu thực nghiệm trình bày trong báo cáo đều do nhóm trực tiếp xây dựng, chạy thử và đo đạc. Những kiến thức, phương pháp hay kết quả kế thừa từ công trình của tác giả khác đều được nhóm ghi rõ nguồn và liệt kê đầy đủ trong mục tài liệu tham khảo. Nhóm khẳng định không sao chép trái phép, không chỉnh sửa số liệu và không sử dụng kết quả của người khác như của mình.

Nếu phát hiện bất kỳ nội dung nào vi phạm cam kết trên, nhóm xin hoàn toàn chịu trách nhiệm trước hội đồng và nhà trường.

Người thực hiện đồ án

LÊ DUY BẢO

TRỊNH PHI HÀO

LỜI CẢM ƠN

Nhóm tác giả xin gửi lời cảm ơn chân thành và sâu sắc đến Thầy Nguyễn Mạnh Hùng đã tận tình hướng dẫn, định hướng nghiên cứu, đóng góp những ý kiến quý báu và luôn tạo điều kiện thuận lợi trong suốt quá trình thực hiện đồ án tốt nghiệp.

Nhờ sự hướng dẫn tận tâm, kiến thức chuyên môn sâu rộng cùng những góp ý thiết thực của Thầy, nhóm đã có thể từng bước hoàn thiện đề tài và đạt được những kết quả nhất định. Đây không chỉ là sự hỗ trợ quý báu trong quá trình thực hiện đồ án mà còn là những kinh nghiệm có giá trị cho công việc và học tập sau này.

Mặc dù nhóm đã nỗ lực hoàn thành đồ án với tinh thần nghiêm túc và trách nhiệm cao, nhưng do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên đồ án khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến nhận xét và đóng góp từ Thầy để đề tài được hoàn thiện hơn.

Kính chúc Thầy sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy và nghiên cứu khoa học.

Nhóm tác giả xin chân thành cảm ơn!

TÓM TẮT

Sự phát triển của Internet Vạn vật (IoT) và nhu cầu xử lý dữ liệu thời gian thực đang đẩy phần tính toán từ đám mây xuống thiết bị biên (Edge AI, TinyML). Tuy nhiên, việc chạy trực tiếp mạng nơ-ron học sâu trên phần cứng hạn chế tài nguyên như FPGA gặp khó khăn về số khối xử lý tín hiệu số (DSP48E1), dung lượng bộ nhớ nội bộ (Block RAM) và tài nguyên logic (LUT/FF).

Đồ án thiết kế và tối ưu hóa bộ gia tốc phần cứng cho mạng Tự mã hóa lượng tử (QAE) trên FPGA Xilinx Artix-7 100T, theo quy trình Đồng thiết kế Phần cứng – Phần mềm (HW/SW Co-design). Ở phần thuật toán, nhóm xây dựng mạng tự mã hóa đối xứng 196-16-8-16-196 và áp dụng Lượng tử hóa nhận thức (QAT) 8-bit bằng thư viện QKeras để mạng hội tụ ổn định và giảm sai số phục dựng. Ở phần phần cứng, mô hình được chuyển sang mã nguồn C++ bằng hls4ml, kiểm chứng logic bằng C-Simulation, rồi tối ưu ở mức thanh ghi (RTL) qua giao thức luồng dữ liệu `io_stream`, định dạng dấu phẩy thập phân `ap_fixed<12,4>` và hệ số tái sử dụng tài nguyên `Reuse Factor = 64`. Toàn bộ được thử nghiệm trên tập dữ liệu MNIST.

Kết quả tổng hợp mạch cho thấy thiết kế sử dụng tài nguyên hợp lý trên dòng chip cấu hình thấp và đạt thông lượng 1.56 triệu ảnh/giây, với $MSE = 0.030$ và $PSNR = 15.18$ dB trên tập kiểm tra MNIST. Bộ gia tốc đạt độ trễ $0.91 \mu s/\text{ảnh}$ (91 chu kỳ tại 100 MHz), nhanh hơn CPU 31.7 lần, công suất 0.665 W và hiệu suất năng lượng 30.68 GOP/s/W trên chip Artix-7 100T. Kết quả này cho thấy việc kết hợp lượng tử hóa nhận thức với tối ưu luồng dữ liệu RTL là khả thi để triển khai mô hình học sâu trên thiết bị biên, và có thể đóng gói thành các lõi IP tái sử dụng cho ứng dụng TinyML.

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
TÓM TẮT	iii
MỤC LỤC.....	iv
DANH MỤC HÌNH.....	vii
DANH MỤC BẢNG	viii
CÁC TỪ VIẾT TẮT	ix
CHƯƠNG 1: GIỚI THIỆU	1
1.1 LÝ DO CHỌN ĐỀ TÀI	1
1.2 MỤC TIÊU NGHIÊN CỨU.....	2
1.2.1 Mục tiêu khoa học	2
1.2.2 Mục tiêu thực tiễn.....	3
1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	3
1.4 PHƯƠNG PHÁP NGHIÊN CỨU.....	4
1.5 Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN	6
1.5.1 Ý nghĩa khoa học.....	6
1.5.2 Ý nghĩa thực tiễn	6
1.6 CẤU TRÚC CỦA KHÓA LUẬN	7
CHƯƠNG 2: NGHIÊN CỨU TỔNG QUÁT.....	9
2.1 TRÍ TUỆ NHÂN TẠO TẠI BIÊN VÀ TINYML	9
2.1.1 Sự dịch chuyển từ Điện toán đám mây sang Điện toán biên	9
2.1.2 Đặc thù khắc nghiệt của Học máy siêu nhỏ	10
2.2 KIẾN TRÚC MẠNG NƠ-RON TỰ MÃ HÓA	12
2.2.1 Cấu trúc tổng quát	12
2.2.2 Phân tích toán học cơ chế nén phi tuyến và nút thắt cổ chai.....	14
2.2.3 Quá trình huấn luyện và Hàm mất mát (MSE).....	14
2.3 LƯỢNG TỬ HÓA TRONG MẠNG NƠ-RON (QUANTIZATION).....	16
2.3.1 Rào cản của dấu phẩy động.....	16
2.3.2 Định dạng số dấu phẩy tĩnh.....	17
2.3.3 Phương pháp Lượng tử hóa nhận thức (QAT) và thư viện QKeras.....	18

2.4 NỀN TẢNG VI MẠCH KHẢ TRÌNH FPGA VÀ DÒNG XILINX ARTIX-7	19
2.4.1 Tổng quan về FPGA.....	19
2.4.2 Kiến trúc tài nguyên chip Artix-7 100T	21
2.5 CÔNG CỤ CHUYỂN ĐỔI HLS VÀ THƯ VIỆN HLS4ML	22
2.5.1 Khái niệm High-Level Synthesis (HLS)	22
2.5.2 Cơ chế biên dịch đồ thị tính toán của thư viện hls4ml.....	23
2.5.3 Các chiến lược tối ưu hóa kiến trúc vi mạch.....	23
CHƯƠNG 3: THIẾT KẾ VÀ TỐI ƯU HÓA HỆ THỐNG.....	25
3.1 SƠ ĐỒ QUY TRÌNH THỰC HIỆN	25
3.2 THIẾT KẾ MÔ HÌNH PHẦN MỀM.....	28
3.2.1 Thu thập và tiền xử lý tập dữ liệu ảnh MNIST	28
3.2.2 Đề xuất kiến trúc mạng Autoencoder tinh gọn	29
3.2.3 Cấu hình tham số lượng tử hóa với QKeras.....	30
3.2.4 Tính toán độ phức tạp mô hình và kích thước bộ nhớ lưu trữ	31
3.3 TỔNG HỢP PHẦN CỨNG VÀ TỐI ƯU HÓA VI KIẾN TRÚC MỨC RTL	33
3.3.1 Quy trình dịch mã C++ tự động hóa bằng hls4ml.....	33
3.3.2 Giải quyết bài toán Routing Congestion bằng giao thức truyền luồng dữ liệu nối tiếp.....	34
3.3.3 Tối ưu hóa toán học bộ cộng dồn tại các nút nơ-ron.....	35
3.4 GIAO THỨC MÔ PHỎNG VÀ KIỂM CHỨNG.....	36
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU VÀ THẢO LUẬN.....	38
4.1 KẾT QUẢ HUẤN LUYỆN VÀ ĐÁNH GIÁ TRÊN PHẦN MỀM.....	38
4.1.1 Đồ thị hội tụ (Loss) và các chỉ số định lượng	38
4.1.2 Bản đồ sai số trực quan	39
4.1.3 Khảo sát thời gian thực thi trên CPU laptop	41
4.2 KẾT QUẢ TỔNG HỢP VÀ GIA TỐC TRÊN PHẦN CỨNG	42
4.2.1 Đánh giá mức độ tiêu thụ tài nguyên vật lý	42
4.2.2 Đánh giá thông số định thời và Tốc độ gia tốc	47
4.2.3 Đánh giá Công suất tiêu thụ	48
4.2.4 Tính toán Thông lượng Toán học.....	50

4.3 ĐỐI CHIẾU HIỆU NĂNG GIỮA FPGA, CPU VÀ GPU	51
4.3.1 Phương pháp đo và cấu hình thực nghiệm	52
4.3.2 Kết quả đo tổng hợp	53
4.3.3 Phân tích nguyên nhân chênh lệch hiệu năng giữa CPU và GPU.....	54
4.3.4 Phân tích theo từng vùng kích thước lô.....	55
4.3.5 Đánh giá ưu thế của FPGA so với CPU và GPU	56
4.4 TIỀM NĂNG TRIỂN KHAI THỰC TẾ.....	56
4.4.1 Định hướng ứng dụng thực tiễn	56
4.4.2 Lợi ích kỹ thuật	57
4.4.3 Chi phí, vận hành và vòng đời	58
4.4.4 Hạn chế khi triển khai	58
4.5 PHÂN TÍCH KIẾN TRÚC QCNN MỞ RỘNG VÀ TÌNH TRẠNG TRÀN TỔNG HỢP FPGA	59
4.5.1 Kiến trúc QCNN ConvAE Mở rộng.....	59
4.5.2 Kết quả Phần mềm - Mô hình chạy được trên CPU.....	59
4.5.3 Phân tích Tình trạng Tràn Tổng hợp FPGA - PRE-SYNTHESIS FAILED	59
4.5.4 Tổng hợp So sánh QAE Dense và QCNN ConvAE	60
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	62
5.1 KẾT LUẬN NHỮNG ĐÓNG GÓP CHÍNH CỦA ĐỀ TÀI	62
5.2 NHỮNG MẶT HẠN CHẾ CÒN TỒN TẠI.....	63
5.3 HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI.....	64
TÀI LIỆU THAM KHẢO	66

DANH MỤC HÌNH

Hình 2.1 Sơ đồ kiến trúc tổng quát và luồng xử lý dữ liệu của mạng nơ-ron Autoencoder.

Hình 2.2 Sơ đồ vi kiến trúc tổng quát của nền tảng vi mạch FPGA.

Hình 2.3 Sơ đồ vi kiến trúc chi tiết của lõi xử lý tín hiệu số DSP48E1 trên FPGA Xilinx 7-Series.

Hình 3.1 Sơ đồ quy trình thực hiện đồ án theo phương pháp đồng thiết kế phần cứng - phần mềm.

Hình 3.2 Sơ đồ biểu diễn phép biến đổi ma trận 2D thành véc-tơ 1D cấp cho lớp nơ-ron của mạng Autoencoder.

Hình 4.1 Biểu đồ đường cong học tập thể hiện sự hội tụ của hàm mất mát (MSE Loss) qua 50 kỷ nguyên huấn luyện.

Hình 4.2 Đánh giá trực quan kết quả suy luận của mô hình QAE trên tập dữ liệu MNIST (Từ trên xuống: Ảnh gốc đầu vào, Ảnh giải mã tái tạo, và Bản đồ nhiệt biểu diễn mức độ sai số).

Hình 4.3 Kết quả triển khai vật lý (Implemented Design) trong Vivado 2022.2 - Màn hình Device View xác nhận thiết kế QAE đã hoàn thành Place & Route thành công trên vi mạch Xilinx Artix-7 (xc7a100tcsg324-1).

DANH MỤC BẢNG

Bảng 2.1 Tổng hợp đối chiếu hiệu năng thực thi mô hình Autoencoder trên các nền tảng phần cứng.

Bảng 4.1 Thống kê mức độ tiêu thụ tài nguyên trên vi mạch Artix-7 (xc7a100tcs324-1).

Bảng 4.2 Thông số ước tính hiệu năng (Performance Estimates) của mô hình mạng tự mã hóa sau khi trải qua luồng tổng hợp mức cao HLS.

Bảng 4.3 Phân bố công suất tiêu thụ động.

Bảng 4.4 Độ trễ theo kích thước lô - nhỏ hơn = tốt hơn ($\mu\text{s}/\text{ảnh}$).

Bảng 4.5 Thông lượng theo kích thước lô - lớn hơn = tốt hơn ($\text{ảnh}/\text{giây}$).

Bảng 4.6 So sánh toàn diện QAE Dense (mô hình gốc) và QCNN ConvAE (mô hình mở rộng).

CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Ý nghĩa/Ứng dụng trong bài
AXI	Advanced eXtensible Interface	Giao thức giao tiếp hệ thống
BRAM	Block RAM	Bộ nhớ tích hợp trên FPGA
DSP	Digital Signal Processing	Khối xử lý tín hiệu số chuyên dụng trên FPGA
FF	Flip-Flop	Phần tử lưu trữ trạng thái tuần tự trong mạch số
FPGA	Field-Programmable Gate Array	Vì mạch khả trình được sử dụng để triển khai hệ thống
FPS	Frames Per Second	Số khung hình xử lý trong một giây
HLS	High-Level Synthesis	Công cụ tổng hợp từ ngôn ngữ C/C++ sang RTL
LUT	Look-Up Table	Bảng tra cứu logic trên FPGA
MAC	Multiply-Accumulate	Phép toán nhân cộng tích lũy
MSE	Mean Squared Error	Hàm mất mát đánh giá sai số tái tạo
PSNR	Peak Signal-to-Noise Ratio	Chỉ số đánh giá chất lượng ảnh tái tạo
PTQ	Post-Training Quantization	Phương pháp lượng tử hóa sau huấn luyện
QAE	Quantized Autoencoder	Mô hình Autoencoder lượng tử hóa của đề tài
QAT	Quantization-Aware Training	Phương pháp huấn luyện có xét đến lượng tử hóa
QCNN	Quantized Convolutional Neural Network	Mạng nơ-ron tích chập lượng tử hóa
RTL	Register Transfer Level	Mức mô tả phần cứng dùng cho tổng hợp mạch
SoC	System-on-Chip	Hệ thống tích hợp trên một vi mạch
VHDL	VHSIC Hardware Description Language	Ngôn ngữ mô tả phần cứng dùng trong thiết kế FPGA

CHƯƠNG 1: GIỚI THIỆU

1.1 LÝ DO CHỌN ĐỀ TÀI

Điện toán biên cho phép trích xuất đặc trưng, xử lý và nén dữ liệu cục bộ ngay tại nơi sinh ra, từ đó tối ưu hóa chi phí băng thông, giảm thiểu độ trễ phản hồi và tăng cường tính an toàn cho hệ thống.

Trong kỷ nguyên Công nghiệp 4.0, sự phát triển nhanh chóng của hệ sinh thái Internet Vạn vật (IoT) đã tạo ra một khối lượng dữ liệu số hóa rất lớn. Theo mô hình Điện toán đám mây (Cloud Computing) truyền thống, toàn bộ nguồn dữ liệu thô thu thập từ các thiết bị đầu cuối, đặc biệt là dữ liệu hình ảnh đa chiều từ hệ thống camera giám sát, cảm biến công nghiệp hay thiết bị y tế, phải được truyền tải liên tục về các trung tâm máy chủ tập trung để xử lý bằng các thuật toán Trí tuệ Nhân tạo (AI). Tuy nhiên, kiến trúc này đang bộc lộ những hạn chế mang tính hệ thống, bao gồm giới hạn về băng thông mạng (Network Bandwidth), độ trễ truyền tải (Latency) không đáp ứng được yêu cầu thời gian thực (Real-time), cùng các rủi ro về bảo mật thông tin. Để giải quyết các thách thức này, xu hướng dịch chuyển năng lực tính toán từ đám mây xuống trực tiếp các thiết bị phần cứng tại biên hay còn gọi là Trí tuệ Nhân tạo tại biên (Edge AI) và Học máy siêu nhỏ (TinyML) đang trở thành một yêu cầu kỹ thuật cấp thiết.

Tuy nhiên, kiến trúc chip Artix-7 có những giới hạn vật lý khắt khe về số lượng khối xử lý tín hiệu số chuyên dụng (DSP48E1), dung lượng bộ nhớ nội bộ (Block RAM) và tài nguyên logic (LUT/FF). Nếu đưa trực tiếp một mạng nơ-ron chưa qua tinh chỉnh cấu trúc từ môi trường phần mềm xuống FPGA, hệ thống phần cứng sẽ đối mặt với nguy cơ vượt quá tài nguyên vật lý (LUT, BRAM, DSP) và hiện tượng nghẽn mạch định tuyến cục bộ (Routing Congestion) do sự gia tăng đột biến của các đường dây định tuyến, dẫn đến việc tổng hợp mạch (Synthesis) thất bại.

Mặc dù mang lại lợi ích lớn về mặt kiến trúc, việc thực thi các mô hình học sâu (Deep Learning) trực tiếp trên thiết bị biên vấp phải rào cản về tài nguyên máy tính. Các mạng nơ-ron học sâu truyền thống thường có cấu trúc phức tạp với hàng

triệu tham số trọng số và được thiết kế để hoạt động dưới định dạng số thực dấu phẩy động 32-bit (float32). Đặc tính này đòi hỏi năng lực tính toán mạnh từ CPU/GPU và dung lượng bộ nhớ lớn, dẫn đến mức tiêu thụ điện năng cao, những điều kiện tạo ra thách thức lớn đối với các thiết bị IoT hoạt động bằng nguồn pin hạn chế. Trong số các nền tảng phần cứng dành cho Edge AI, vi mạch khả trình FPGA, điển hình là dòng chip Xilinx Artix-7, là một giải pháp gia tốc hiệu quả nhờ kiến trúc xử lý song song, độ trễ rất thấp và hiệu suất năng lượng cao.

Để thu hẹp khoảng cách giữa yêu cầu tính toán của mô hình và tài nguyên phần cứng, đề án áp dụng quy trình Đồng thiết kế Phần cứng - Phần mềm (HW/SW Co-design): thu gọn kiến trúc mạng, lượng tử hóa trọng số từ float32 xuống 8-bit qua QAT, và thiết kế lại luồng dữ liệu RTL để chia sẻ tài nguyên hiệu quả hơn. Từ đó, đề tài "Thiết kế và tối ưu hóa Quantized Autoencoder trên FPGA Artix-7 cho ứng dụng nén ảnh MNIST" được lựa chọn để nghiên cứu và kiểm chứng.

1.2 MỤC TIÊU NGHIÊN CỨU

1.2.1 Mục tiêu khoa học

Đề án tập trung nghiên cứu, thiết kế và triển khai bộ gia tốc phần cứng cho mạng Tự mã hóa lượng tử (QAE) trên nền tảng FPGA Xilinx Artix-7 100T thông qua quy trình đồng thiết kế phần cứng - phần mềm. Mục tiêu đầu tiên là thiết kế và tối ưu hóa kiến trúc, trong đó tập trung xây dựng mô hình mạng tự mã hóa đối xứng (196-16-8-16-196) và áp dụng phương pháp Lượng tử hóa nhận thức (QAT) qua thư viện QKeras để chuyển đổi định dạng ma trận trọng số sang chuẩn số nguyên 8-bit, giúp đảm bảo độ hội tụ và giảm thiểu sai số phục dựng. Tiếp theo, đề án thực hiện chuyển đổi và kiểm chứng thiết kế bằng cách sử dụng công cụ hls4ml để biên dịch cấu trúc mô hình từ file .h5 sang mã nguồn C++, kết hợp với việc thực hiện kiểm chứng logic thông qua mô phỏng phần mềm (C-Simulation) nhằm xác nhận tính đúng đắn về mặt toán học. Cuối cùng, đề án hướng tới việc tối ưu hóa vi kiến trúc RTL thông qua các giải pháp kỹ thuật chuyên sâu như chuyển đổi giao thức truyền dẫn (io_stream), cấu hình định dạng dấu phẩy tính (ap_fixed<12,4>) và thiết lập hệ số tái sử dụng tài nguyên (Reuse Factor = 64) để cân bằng tài nguyên trên FPGA.

1.2.2 Mục tiêu thực tiễn

Thông qua việc hiện thực hóa các giải pháp kỹ thuật nêu trên, đề án hướng tới việc giải quyết bài toán Edge AI bằng cách cung cấp giải pháp nén và hạ chiều dữ liệu hình ảnh trực tiếp tại thiết bị biên, giúp khắc phục hiệu quả các rào cản về băng thông và độ trễ trong các ứng dụng thực tế. Bên cạnh đó, đề án còn đóng góp vào việc phát triển các hệ thống học máy có khả năng vận hành ổn định trên các nền tảng phần cứng hạn chế về tài nguyên như TinyML. Đồng thời, kết quả nghiên cứu cũng là minh chứng cho tính khả thi của hệ thống thông qua việc tối ưu hóa đồng bộ ba tiêu chí kỹ thuật then chốt, bao gồm hiệu quả sử dụng tài nguyên logic (LUT, BRAM, DSP48E1), duy trì thông lượng (Throughput) cao đáp ứng yêu cầu thời gian thực và đảm bảo độ chính xác phục dựng tín hiệu trong ngưỡng kiểm soát.

1.3 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Để hiện thực hóa quy trình Đồng thiết kế Phần cứng - Phần mềm (HW/SW Co-design) cho các ứng dụng Trí tuệ Nhân tạo tại biên (Edge AI), đề án nghiên cứu đồng bộ các thành phần cốt lõi trong toàn bộ vòng đời thiết kế từ thuật toán đến vi kiến trúc phần cứng.

Đối tượng nghiên cứu trọng tâm là kiến trúc mạng nơ-ron Tự mã hóa (Autoencoder) cùng cơ chế toán học trong bài toán hạ chiều không gian và nén dữ liệu hình ảnh không giám sát. Việc phân tích cấu trúc các hàm ánh xạ phi tuyến tính bên trong bộ mã hóa (Encoder) cho phép tối ưu hóa khả năng rút gọn dữ liệu từ không gian đặc trưng ban đầu xuống không gian ẩn (Latent Space) có số chiều tối giản, đảm bảo tái cấu trúc chính xác ma trận điểm ảnh tại bộ giải mã (Decoder). Đây chính là nền tảng toán học định hình đồ thị tính toán (Computation Graph) cho toàn bộ hệ thống gia tốc phần cứng.

Từ nền tảng thuật toán này, đề tài nghiên cứu kỹ thuật Lượng tử hóa nhận thức (QAT) để hiện thực hóa việc ép kiểu dữ liệu. Phương pháp này tích hợp nhiều làm tròn vào pha lan truyền xuôi ngay trong quá trình huấn luyện, kết hợp với các bộ ước lượng đạo hàm (Straight-Through Estimator) để cập nhật trọng số. Mục

tiêu là xác định phương pháp thu hẹp không gian biểu diễn của các ma trận tham số cùng giá trị kích hoạt từ định dạng số thực 32-bit (float32) sang định dạng số nguyên 8-bit, đảm bảo mô hình vẫn duy trì tính hội tụ ổn định trước khi chuyển dịch sang cấu trúc mạch điện.

Để chuyển đổi thành công mô hình đã lượng tử hóa xuống vi mạch, nghiên cứu sử dụng luồng thiết kế biên dịch cấp cao (HLS) thông qua công cụ hls4ml. Việc phân tích cấu trúc mạng bằng hls4ml cho phép tự động chuyển đổi mô hình sang mã nguồn C++, làm cơ sở để ánh xạ các phép toán nhân chập vào bốn nguồn tài nguyên cốt lõi trên chip Artix-7: bảng tra cứu logic (LUT), thanh ghi trạng thái (Flip-Flop), bộ nhớ tĩnh cục bộ (Block RAM) và các khối xử lý tín hiệu số chuyên dụng (DSP48E1). Những kết quả phân tích về vận hành vật lý này là tiền đề quyết định để đề xuất chiến lược tối ưu hóa mức thanh ghi (RTL), ngăn ngừa tình trạng tràn diện tích và nghẽn mạch định tuyến trên linh kiện.

Về phạm vi và giới hạn, đề án khoanh vùng trên bốn phương diện chính: dữ liệu thử nghiệm, công cụ phát triển, nền tảng phần cứng và giới hạn kiểm chứng. Cụ thể, đề tài sử dụng bộ dữ liệu chuẩn MNIST (ảnh chữ số viết tay 28x28 pixel). Quá trình thiết kế được thực hiện trên Python (TensorFlow/Keras, QKeras), thư viện hls4ml (phiên bản 1.3.0) và bộ công cụ Xilinx Vitis HLS (phiên bản 2022.2). Nền tảng phần cứng mục tiêu là chip Xilinx Artix-7 100T (xc7a100tcs324-1) với các ràng buộc khắt khe về LUT, FF, BRAM và 240 khối DSP48E1. Cuối cùng, Phạm vi kiểm chứng bao gồm C-Simulation, Synthesis và Implementation (Place & Route) trong môi trường phần mềm Vivado 2022.2. Đề tài không thực hiện nạp bitstream lên bo mạch phần cứng thực tế.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Để giải quyết bài toán tối ưu hóa và triển khai thuật toán học sâu trên nền tảng phần cứng hạn chế tài nguyên, đề án áp dụng phương pháp nghiên cứu phối hợp giữa phân tích lý thuyết, thực nghiệm mô phỏng và đánh giá định lượng. Toàn bộ quy trình được vận hành đồng bộ theo luồng Đồng thiết kế Phần cứng - Phần mềm (HW/SW Co-design Flow).

Nền tảng khoa học của đề án được xây dựng thông qua việc tổng hợp, kế thừa và phân tích chuyên sâu các tài liệu học thuật và công trình nghiên cứu uy tín. Nghiên cứu tập trung vào cơ chế toán học của mạng Tự mã hóa (Autoencoder) trong bài toán hạ chiều phi tuyến, nguyên lý hoạt động của kỹ thuật Lượng tử hóa nhận thức (QAT) chuẩn 8-bit, và cấu trúc hình học vật lý của vi mạch khả trình Xilinx Artix-7. Việc phân tích lý thuyết giúp định hình rõ ràng các ràng buộc phần cứng, cơ chế định tuyến dữ liệu nội bộ và khả năng phân bổ năng lực toán học của các khối xử lý tín hiệu chuyên dụng (DSP48E1) hay bộ nhớ nội bộ (Block RAM), làm cơ sở lý luận để đề xuất các chiến lược tối ưu hóa vi kiến trúc.

Dựa trên nền tảng đó, phương pháp nghiên cứu thực nghiệm định lượng theo tính chất lặp (iterative process) đóng vai trò chủ đạo trong việc hiện thực hóa, kiểm chứng và tinh chỉnh hệ thống. Tại phần phần mềm, các mô hình Autoencoder với những cấu hình tham số khác nhau được huấn luyện trực tiếp bằng thư viện QKeras nhằm xác định điểm hội tụ tối ưu cho định dạng số nguyên bit thấp. Tiến tới pha phần cứng, quy trình mô phỏng lặp được áp dụng nghiêm ngặt trong môi trường Xilinx Vitis HLS. Bằng việc thực thi xoay vòng các bước mô phỏng thuật toán cấp cao (C-Simulation) và tổng hợp mạch logic (C/RTL Synthesis), nghiên cứu chủ động phát hiện sớm các rủi ro tràn tài nguyên hoặc nghẽn mạch vật lý (Routing Congestion). Từ các thông số cảnh báo, quy trình cho phép kỹ sư quay ngược lại cấu trúc mã nguồn để điều chỉnh linh hoạt các thông số vi kiến trúc cốt lõi như giao thức luồng dữ liệu (io_stream), hệ số tái sử dụng tài nguyên (Reuse Factor) hay độ rộng bit của bộ cộng dồn.

Giai đoạn cuối cùng sử dụng phương pháp so sánh và phân tích đối chiếu nhằm khẳng định tính khả thi của các giải pháp kỹ thuật. Các phép đo đặc được thực hiện thông qua việc đối chiếu trực tiếp chất lượng hình ảnh phục dựng sau nén bằng chỉ số Sai số toàn phương trung bình (MSE) và Tỷ số tín hiệu cực đại trên nhiễu (PSNR) giữa mô hình mô phỏng phần cứng và nguyên mẫu phần mềm. Song song đó, các số liệu trích xuất từ báo cáo phân bổ tài nguyên vật lý (LUT, Flip-Flop, BRAM, DSP) và thông số định thời được phân tích minh bạch. Sự đối chiếu này nhằm minh chứng định lượng cho khả năng tối ưu hóa diện tích mạch

trên dòng chip cấu hình thấp, đồng thời bảo toàn năng suất xử lý dữ liệu thời gian thực.

1.5 Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN

1.5.1 Ý nghĩa khoa học

Về mặt khoa học, đề án đóng góp một tài liệu tham khảo chuyên sâu và hệ thống về quy trình Đồng thiết kế Phần cứng - Phần mềm (Hardware/Software Co-design) - một phương pháp luận quan trọng và ngày càng phổ biến trong lĩnh vực thiết kế vi mạch chuyên dụng hiện nay. Nghiên cứu minh chứng rõ nét tầm quan trọng mang tính quyết định của việc thấu hiểu các ràng buộc vật lý trên cấu trúc vi mạch (như hiện tượng nghẽn mạch định tuyến cục bộ, giới hạn tài nguyên DSP48E1, Block RAM và LUT) đối với việc định hướng, tinh chỉnh vi cấu trúc của thuật toán học máy ngay từ tầng phần mềm.

Thay vì xem phần cứng và thuật toán là hai thực thể độc lập, kết quả nghiên cứu thiết lập một tư duy thiết kế đồng bộ mang tính hai chiều. Đề tài chứng minh rằng việc kết hợp chặt chẽ giữa kỹ thuật lượng tử hóa nhận thức (QAT) chuẩn 8-bit trên phần mềm với kiến trúc luồng dữ liệu mức thanh ghi (RTL) trên phần cứng là cơ sở để triển khai các mô hình học sâu như mạng Tự mã hóa (Autoencoder) trên các nền tảng thiết bị biên có tài nguyên hạn chế. Sự tối ưu hóa này đảm bảo hệ thống vận hành ổn định mà không làm suy giảm nghiêm trọng độ chính xác toán học. Đây là đóng góp có ý nghĩa đối với các nghiên cứu tối ưu hóa kiến trúc máy tính và gia tốc phần cứng cho các thuật toán trí tuệ nhân tạo.

1.5.2 Ý nghĩa thực tiễn

Về mặt thực tiễn, toàn bộ quy trình thiết kế, biên dịch cùng bộ thông số cấu hình vi kiến trúc tối ưu được đúc kết từ đề án có khả năng đóng gói hoàn chỉnh thành các mô-đun sở hữu trí tuệ (IP Cores) chuẩn mực. Các lõi IP này sở hữu tính linh hoạt và khả năng tái sử dụng cao, sẵn sàng để tích hợp trực tiếp vào các hệ thống trên chip (SoC) hoặc các dự án thiết kế mạch phức tạp trong công nghiệp bán dẫn. Kết quả nghiên cứu tạo ra một tiền đề kỹ thuật khả thi, thúc đẩy việc triển khai các hệ thống học máy siêu nhỏ (TinyML) tiến sát đến các thiết bị đầu cuối hoạt động ngoài môi trường thực tế.

Kiến trúc Quantized Autoencoder tối ưu này có tiềm năng ứng dụng trực tiếp vào nhiều lĩnh vực, từ chế tạo các hệ thống camera thông minh sử dụng thị giác máy tính để giám sát và phân loại sản phẩm tự động tại nhà máy, đến việc phát triển các thiết bị y tế đeo được có khả năng theo dõi sức khỏe bệnh nhân liên tục. Ngoài ra, mô hình còn có thể vận hành trên các nút trạm thu thập tín hiệu cảm biến trong mạng lưới IoT diện rộng. Việc thực thi tác vụ trích xuất đặc trưng và nén dữ liệu tại biên giúp giải quyết hiệu quả bài toán thất cổ chai về băng thông truyền tải, giảm thiểu đáng kể độ trễ so với các mô hình đám mây truyền thống, đồng thời đáp ứng các tiêu chí về tối ưu hóa chi phí linh kiện phần cứng và tiết kiệm năng lượng tối đa cho các thiết bị nhúng dùng pin. Qua đó, đề án cung cấp một giải pháp khả thi cho các hệ thống đòi hỏi tính tự chủ cao và khả năng vận hành bền bỉ trong môi trường công nghiệp thực tế.

1.6 CẤU TRÚC CỦA KHÓA LUẬN

Nội dung của khóa luận được phân bổ thành năm chương chính, tuân thủ chặt chẽ trình tự logic từ cơ sở lý thuyết, quy trình thiết kế cho đến kết quả thực nghiệm và định hướng phát triển.

Chương 1, "Giới thiệu", thiết lập nền tảng khoa học cho đề tài thông qua việc phân tích bối cảnh cấp thiết của lĩnh vực Trí tuệ Nhân tạo tại biên (Edge AI) và sự cần thiết của quy trình Đồng thiết kế Phần cứng - Phần mềm (HW/SW Co-design). Chương này cũng xác định rõ ràng mục tiêu, đối tượng, phạm vi nghiên cứu và phương pháp luận được áp dụng xuyên suốt khóa luận.

Chương 2, "Nghiên cứu tổng quát", tập trung hệ thống hóa các kiến thức nền tảng về mạng nơ-ron Tự mã hóa (Autoencoder), cơ chế toán học của kỹ thuật Lượng tử hóa nhận thức (QAT), và kiến trúc vi mạch khả trình FPGA Xilinx Artix-7. Nội dung chương này cung cấp cơ sở lý luận để hiểu rõ các ràng buộc vật lý và các kỹ thuật tối ưu hóa sẽ được áp dụng trong các chương tiếp theo.

Chương 3, "Thiết kế và tối ưu hóa hệ thống", đi sâu vào quy trình hiện thực hóa giải pháp. Chương này trình bày chi tiết về quá trình xây dựng kiến trúc mạng, thiết lập mô hình lượng tử hóa trên phần mềm, và quy trình chuyển dịch đồ thị tính toán thông qua công cụ hls4ml. Trọng tâm của chương là các chiến lược can thiệp

vào vi kiến trúc mức thanh ghi (RTL) để tối ưu hóa việc phân bổ tài nguyên phần cứng trên chip.

Chương 4, "Kết quả nghiên cứu và thảo luận", công bố các kết quả thực nghiệm thu được từ quá trình mô phỏng thuật toán cấp cao và tổng hợp mạch logic. Nội dung chương tập trung vào việc đối chiếu chất lượng hình ảnh phục dựng, đánh giá hiệu quả sử dụng tài nguyên vật lý và các thông số định thời. Qua đó, chương này khẳng định tính khả thi của giải pháp đề xuất thông qua các chỉ số định lượng cụ thể.

Chương 5, "Kết luận và hướng phát triển", tổng kết các đóng góp chính của đồ án về mặt khoa học và thực tiễn. Chương này cũng nhìn nhận lại những giới hạn của nghiên cứu và đề xuất các hướng phát triển tiếp theo để nâng cao hiệu suất cũng như mở rộng khả năng ứng dụng của mô hình trong môi trường công nghiệp thực tế.

CHƯƠNG 2: NGHIÊN CỨU TỔNG QUÁT

2.1 TRÍ TUỆ NHÂN TẠO TẠI BIÊN VÀ TINYML

2.1.1 Sự dịch chuyển từ Điện toán đám mây sang Điện toán biên

Trong suốt thập kỷ qua, kiến trúc Điện toán đám mây (Cloud Computing) đóng vai trò là nền tảng cốt lõi cho việc triển khai các ứng dụng Trí tuệ Nhân tạo nhờ khả năng cung cấp tài nguyên tính toán và lưu trữ gần như vô hạn. Tuy nhiên, khi hệ sinh thái Internet Vạn vật (IoT) bùng nổ với hàng tỷ thiết bị đầu cuối liên tục sản sinh dữ liệu thô ở quy mô lớn, mô hình xử lý tập trung này bắt đầu bộc lộ những điểm nghẽn kỹ thuật nghiêm trọng.

Điểm nghẽn đầu tiên nằm ở băng thông đường truyền (Network Bandwidth). Việc truyền tải liên tục luồng dữ liệu hình ảnh độ phân giải cao từ hàng ngàn camera giám sát về máy chủ trung tâm gây ra áp lực quá tải nghiêm trọng cho hạ tầng mạng. Thứ hai, yếu tố độ trễ (Latency) của mạng truyền thông không thể đảm bảo tính tất định, gây rủi ro lớn cho các ứng dụng đòi hỏi phản hồi thời gian thực như xe tự hành, hỗ trợ y tế khẩn cấp hoặc giám sát dây chuyền sản xuất tự động. Cuối cùng, rủi ro về an ninh mạng, rò rỉ thông tin riêng tư và chi phí vận hành hệ thống máy chủ khổng lồ là những thách thức chưa có lời giải thỏa đáng trong mô hình đám mây cũ.

Để khắc phục các hạn chế trên, ngành kiến trúc máy tính đã dịch chuyển mạnh mẽ sang mô hình Điện toán biên (Edge AI). Về bản chất khoa học, Edge AI là quá trình phân tán năng lực tính toán và thực thi các thuật toán học máy từ các trung tâm dữ liệu đám mây xuống trực tiếp các nút phần cứng nằm tại biên của mạng lưới - nơi dữ liệu thực tế được sinh ra.

Bằng cách tích hợp các bộ gia tốc phần cứng chuyên dụng ngay tại thiết bị đầu cuối, dữ liệu thô (như ma trận điểm ảnh từ cảm biến thị giác) sẽ được tiền xử lý, trích xuất đặc trưng và nén cục bộ. Quá trình này giúp rút ngắn thời gian truyền tải dữ liệu thô qua mạng, giảm độ trễ phản hồi xuống mức mili giây, tối ưu hóa băng thông bằng cách chỉ gửi các thông tin đã được tinh gọn (hoặc các đoạn mã rất nhỏ) về trung tâm, đồng thời cô lập dữ liệu tại chỗ nhằm bảo vệ quyền riêng tư cho hệ thống [1].

2.1.2 Đặc thù khắc nghiệt của Học máy siêu nhỏ

Tiến xa hơn một bước trong thang phân cấp của Điện toán biên, khái niệm Học máy siêu nhỏ (TinyML - Tiny Machine Learning) xuất hiện như một nhánh nghiên cứu chuyên biệt và khắc nghiệt nhất [2]. Trong khi Edge AI nói chung vẫn có thể triển khai trên các hệ thống máy tính nhúng phân khúc cao sở hữu bộ xử lý đồ họa di động (Mobile GPU) hoặc máy chủ thu nhỏ với công suất tiêu thụ từ vài Watt đến hàng chục Watt, thì TinyML hướng thẳng đến phân khúc các thiết bị vi điều khiển (MCU) hoặc vi mạch khả trình (FPGA) cấu hình thấp. Đặc thù lớn nhất định hình nên TinyML là các giới hạn vật lý và ràng buộc tài nguyên cực kỳ nghiêm ngặt của phần cứng mục tiêu.

Ràng buộc về mặt năng lượng thể hiện ở việc các thiết bị TinyML thường hoạt động trong điều kiện biệt lập, không kết nối lưới điện và phải vận hành liên tục trong nhiều năm chỉ với một viên pin cúc áo hoặc nguồn năng lượng thu hoạch tự nhiên. Do đó, công suất tiêu thụ định mức của toàn hệ thống bắt buộc phải duy trì ở ngưỡng cực thấp, chỉ được tính bằng mili-Watt (mW) hoặc micro-Watt (μ W).

Ràng buộc về năng lực lưu trữ và bộ nhớ nội bộ đặt ra một rào cản vật lý trực tiếp. Trái ngược với không gian lưu trữ rộng lớn trên máy chủ, các chip xử lý TinyML chỉ sở hữu dung lượng bộ nhớ tĩnh (SRAM) và bộ nhớ chương trình (Flash/Block RAM) vô cùng hạn hẹp, dao động từ vài chục Kilo-Byte đến vài Mega-Byte, ngăn chặn việc lưu trữ ma trận tham số khổng lồ của các mạng nơ-ron học sâu truyền thống.

Ràng buộc về năng lực tính toán số học hình thành từ việc các bộ xử lý nhúng phân khúc thấp thường thiếu các đơn vị tính toán số thực dấu phẩy động (FPU) hiệu năng cao. Khi không có FPU, mỗi phép MAC trên float32 phải giả lập bằng phần mềm nên tốn nhiều chu kỳ xung nhịp và nhiều năng lượng.

Chính những rào cản vật lý khắt khe này đã khẳng định tính tất yếu của quy trình Đồng thiết kế Phần cứng - Phần mềm (HW/SW Co-design) trong TinyML. Để một mạng nơ-ron phức tạp như mạng Tự mã hóa (Autoencoder) có thể nhúng và vận hành ổn định tại biên, kỹ sư bắt buộc phải áp dụng các kỹ thuật tinh chỉnh thuật toán chuyên sâu như thu gọn cấu trúc hình học của mạng, cắt tỉa nơ-ron dư thừa (Pruning), và đặc biệt là kỹ thuật lượng tử hóa (Quantization) để ép kiểu dữ

liệu về các định dạng số nguyên có độ rộng bit thấp (như 8-bit hoặc cấu trúc dấu phẩy tĩnh `ap_fixed`). Việc ánh xạ cấu trúc toán học phần mềm sang kiến trúc logic phần cứng chính là yếu tố then chốt để hiện thực hóa các nút cảm biến nhận thức thông minh trong thực tế.

Để chứng minh một cách định lượng cho xu hướng dịch chuyển kiến trúc và làm rõ rào cản năng lượng, độ trễ trong môi trường phần cứng biên, Bảng 2.1 dưới đây tổng hợp kết quả đối chiếu hiệu năng thực thi mô hình Autoencoder trên các nền tảng phần cứng khác nhau được trích xuất từ nghiên cứu của Murat Isik và các cộng sự [3].

Bảng 2.1 Tổng hợp đối chiếu hiệu năng thực thi mô hình Autoencoder trên các nền tảng phần cứng [3].

Platform	Technology	Operation Frequency	Power	Latency	Throughput	Energy Efficiency
NVIDIA K80	ASIC (28nm)	1.48GHz	30W	1137.62ms	22000(GOP/s)	733.33(GOP/s/W)
NVIDIA GTX 1080 TI	GPU(16nm)	1.48GHz	250W	6.15ms	235.77(GOP/s)	0.94(GOP/s/W)
Chakradhar et al.	FPGA(28nm)	200MHz	15W	—	16(GOP/s)	1.06(GOP/s/W)
Gokhale et al	FPGA(28nm)	150MHz	8W	4.50ms	23.18(GOP/s)	2.90(GOP/s/W)
Zhang et al	FPGA(28nm)	100MHz	18.61W	21.61ms	61.62(GOP/s)	3.31(GOP/s/W)
Murat Isik và cs	FPGA(16nm)	100MHz	5.93W	2.91ms	21.12(GOP/s)	3.56(GOP/s/W)

Dựa trên các số liệu định lượng từ Bảng 2.1, có thể thấy rõ sự đánh đổi sâu sắc giữa năng suất tính toán gộp và công suất tiêu thụ điện năng. Kiến trúc đồ họa truyền thống như NVIDIA GTX 1080 TI tuy cung cấp năng suất xử lý cao (235.77 GOP/s) và độ trễ thấp (6.15 ms), nhưng mức tiêu thụ công suất lên đến 250 W đã

loại bỏ nền tảng này khỏi các ứng dụng TinyML dùng pin tại biên. Ngược lại, các giải pháp dựa trên kiến trúc vi mạch khả trình FPGA thể hiện ưu thế vượt trội về khả năng kiểm soát dòng năng lượng.

Đáng chú ý, cấu trúc phần cứng được đề xuất bởi Isik et al. vận hành ở tần số thấp 100 MHz đã tối ưu hóa công suất tiêu thụ xuống ngưỡng 5.93 W, đồng thời đạt độ trễ phản hồi cực thấp là 2.91 ms. Khi xét trên thước đo quan trọng nhất của hệ thống biên là Hiệu suất năng lượng (đo bằng số tỷ phép tính thực hiện được trên một Watt điện năng tiêu thụ - GOP/s/W), kiến trúc FPGA của Isik et al. đạt 3.56 GOP/s/W, cao gấp gần 4 lần so với nền tảng GPU [3].

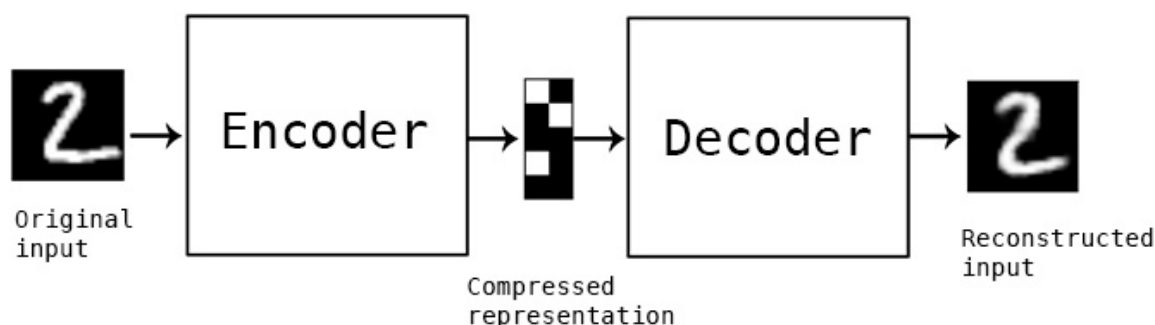
Kết quả thực nghiệm này khẳng định luận điểm khoa học cốt lõi: các hệ thống TinyML không tìm kiếm những bộ cấu trúc có năng suất xử lý thô lớn nhất bằng mọi giá, mà ưu tiên tối ưu hóa mật độ toán học trên mỗi đơn vị năng lượng hao tổn. Đây là cơ sở lý giải cho việc lựa chọn vi mạch FPGA Xilinx Artix-7 kết hợp giải pháp lượng tử hóa dấu phẩy thập phân thấp trong đề án nhằm đạt được điểm cân bằng vi kiến trúc tối ưu.

2.2 KIẾN TRÚC MẠNG NƠ-RON TỰ MÃ HÓA

2.2.1 Cấu trúc tổng quát

Mạng nơ-ron Tự mã hóa (Autoencoder) là một cấu trúc học sâu hoạt động theo cơ chế tự giám sát (self-supervised learning) [4]. Đặc trưng của kiến trúc này là dùng chính dữ liệu ngõ vào làm nhãn mục tiêu cho ngõ ra, nhờ đó không phụ thuộc vào tập nhãn gán thủ công vốn tốn kém. Mục tiêu của mạng là học cách biểu diễn dữ liệu một cách tự động: thông tin kích thước lớn được đưa qua một vùng giới hạn để loại bỏ thành phần nhiễu và dư thừa, chỉ giữ lại những đặc trưng cơ bản nhất dưới dạng một không gian ẩn có số chiều nhỏ.

Một mô hình Autoencoder tiêu chuẩn gồm ba thành phần liên kết theo một chuỗi pipeline tuyến tính: bộ mã hóa (Encoder), không gian ẩn (Latent Space) hay nút thắt cổ chai (Bottleneck), và bộ giải mã (Decoder). Ba khối này tạo thành một chu trình khép kín đi từ hạ chiều dữ liệu, mã hóa thông tin, đến khôi phục lại ma trận dữ liệu ban đầu, như minh họa trong Hình 2.1.



Hình 2.1 Sơ đồ kiến trúc tổng quát và luồng xử lý dữ liệu của mạng nơ-ron Autoencoder.

Bộ mã hóa tiếp nhận ma trận dữ liệu gốc có độ phân giải cao và số chiều lớn từ cảm biến thị giác hoặc camera đầu cuối. Thông qua một chuỗi phép biến đổi đại số tuyến tính (nhân ma trận trọng số và cộng véc-tơ độ lệch) kết hợp với các hàm kích hoạt phi tuyến xếp tuần tự, bộ mã hóa thực hiện việc nén và hạ chiều dữ liệu. Với ảnh thang độ xám, các lớp nơ-ron trong bộ mã hóa loại bỏ những vùng dư thừa (như phần nền đen không chứa thông tin trong ảnh chữ số) và biểu diễn các đường nét, góc cạnh hay độ cong của nét về thành các tọa độ toán học mới có dung lượng nhỏ hơn.

Không gian ẩn (hay lớp thắt cổ chai) là trung tâm của kiến trúc, đúng như phần "compressed representation" ở giữa sơ đồ Hình 2.1. Đây là tầng nơ-ron có kích thước nhỏ nhất trong toàn mạng, chứa véc-tơ đặc trưng (latent representation vector) đại diện cho bản tóm tắt nén của ảnh ngõ vào. Độ rộng của lớp này quyết định trực tiếp hệ số nén và chất lượng phục dựng: số nơ-ron tại đây càng nhỏ thì tài nguyên bộ nhớ (Block RAM) trên vi mạch càng được tiết kiệm, qua đó tối ưu không gian đệm cho phần cứng biên.

Bộ giải mã hoàn thiện chu trình. Khối này tiếp nhận véc-tơ đặc trưng từ không gian ẩn rồi giải mã, mở rộng và phục dựng lại ma trận dữ liệu ban đầu. Kiến

trúc của bộ giải mã thường đối xứng qua gương so với bộ mã hóa nhưng hoạt động theo chiều ngược lại, dùng các lớp nơ-ron mở rộng dần để tái tạo không gian hình học gốc. Đầu ra là một ma trận điểm ảnh có cùng kích thước với ngõ vào. Mức độ giống nhau giữa ảnh phục dựng và ảnh gốc là tiêu chí định lượng chính để đánh giá năng lực trích xuất thông tin của toàn bộ quy trình đồng thiết kế.

2.2.2 Phân tích toán học cơ chế nén phi tuyến và nút thắt cổ chai

Cơ chế vận hành của nút thắt cổ chai trong việc hạ chiều dữ liệu được định nghĩa chặt chẽ thông qua các hàm toán học phi tuyến [5]. Nếu gọi véc-tơ điểm ảnh ngõ vào là $x \in \mathbb{R}^d$, bộ mã hóa sẽ thực hiện một hàm ánh xạ f để chuyển đổi cấu trúc mảng của x thành một véc-tơ biểu diễn tiềm ẩn $h \in \mathbb{R}^m$ tại không gian ẩn thông qua phương trình:

$$h = f(W_e x + b_e) \quad (2.1)$$

Trong đó, W_e đại diện cho ma trận trọng số (weights) của bộ mã hóa, b_e là véc-tơ hệ số độ lệch (biases), và f là hàm kích hoạt phi tuyến tính (như hàm ReLU). Ràng buộc vật lý khắt khe đặt ra để mạng thực hiện chức năng nén dữ liệu là số chiều của không gian ẩn phải nhỏ hơn rất nhiều so với không gian ngõ vào ban đầu ($m \ll d$). Trong kiến trúc được đề xuất của đồ án, ma trận điểm ảnh ngõ vào sau khi làm phẳng có kích thước $d = 196$ sẽ bị ép qua một nút thắt cổ chai có số chiều cực hạn là $m = 8$.

Tính chất phi tuyến tính được cung cấp bởi các hàm kích hoạt giúp Autoencoder vượt trội hơn hẳn so với các phương pháp hạ chiều dữ liệu tuyến tính cổ điển như Phân tích thành phần chính (PCA). Trong khi PCA chỉ tìm kiếm các phép chiếu để tối đa hóa phương sai trên các trục tọa độ tuyến tính, Autoencoder có khả năng học được các mặt cong đa tạp (manifolds) phức tạp. Sự ưu việt này cho phép hệ thống nén chặt dữ liệu hình ảnh với hệ số cao nhưng vẫn bảo toàn được các thông tin cốt lõi, giảm thiểu tối đa tải trọng lưu trữ cho các khối bộ nhớ tĩnh hạn hẹp trên cấu trúc chip Artix-7.

2.2.3 Quá trình huấn luyện và Hàm mất mát (MSE)

Quá trình huấn luyện mạng Tự mã hóa (Autoencoder) được thực thi thông qua chu trình lặp tuần tự của thuật toán lan truyền ngược (Backpropagation) và các

thuật toán tối ưu hóa (như Adam hoặc SGD) [4]. Trọng tâm của quá trình này là việc định lượng sai số giữa dữ liệu đầu vào và dữ liệu được phục dựng để cập nhật trọng số.

Trong pha lan truyền xuôi (forward propagation), bộ giải mã nhận véc-tơ đặc trưng tiềm ẩn h từ nút thắt cổ chai và thực hiện hàm ánh xạ ngược g để tạo ra véc-tơ phục dựng $\hat{x} \in \mathbb{R}^d$ ở ngõ ra:

$$\hat{x} = g(W_d h + b_d) \quad (2.2)$$

Trong đó, W_d và b_d lần lượt là ma trận trọng số và véc-tơ độ lệch của các tầng giải mã. Sự sai lệch biên độ giữa giá trị điểm ảnh thực tế ban đầu x và giá trị điểm ảnh do mạng dự đoán $\hat{x}$ sẽ được tính toán thông qua hàm Sai số toàn phương trung bình (Mean Squared Error - MSE). Trong kiến trúc Autoencoder, MSE đóng vai trò là hàm mất mát (Loss Function) tiêu chuẩn để đánh giá chất lượng khôi phục ảnh:

$$MSE = \frac{1}{N} \sum (x_i - \hat{x}_i)^2 \text{ (Với } i \text{ chạy từ 1 đến } N) \quad (2.3)$$

Trong phương trình toán học trên:

N đại diện cho tổng số phần tử không gian của một mẫu dữ liệu. Đối với ma trận ảnh đầu vào đã được làm phẳng (flatten) trong đồ án, kích thước không gian được cố định ở mức $N = 196$ điểm ảnh.

x_i là cường độ của điểm ảnh thô nguyên bản tại vị trí thứ i , với giá trị đã được chuẩn hóa toán học về khoảng $[0, 1]$.

$\hat{x}_i$ là cường độ của điểm ảnh tương ứng do bộ giải mã phục dựng lại tại cùng vị trí thứ i .

Vì lấy bình phương sai số, MSE phạt nặng những điểm ảnh có sai lệch lớn. Cơ chế toán học này đặc biệt hiệu quả đối với ảnh chữ số viết tay MNIST, bởi nó ép buộc mạng phải tập trung ưu tiên học và bảo toàn các cấu trúc hình học cốt lõi (như nét vẽ, độ cong, góc cạnh của chữ số) thay vì lãng phí tài nguyên để tối ưu hóa vùng nền đen xám tĩnh.

Trong bối cảnh của quy trình Đồng thiết kế Phần cứng - Phần mềm (HW/SW Co-design), việc giám sát sự hội tụ của hàm mất mát MSE không chỉ mang ý nghĩa đánh giá thuật toán thông thường. Giá trị MSE tối thiểu đạt được trên môi trường phần mềm (TensorFlow/Keras) sẽ đóng vai trò là "đường cơ sở"

(baseline) định lượng vô cùng khắt khe. Đường cơ sở này được dùng để đối chiếu và kiểm chứng mức độ suy hao thông tin vật lý khi kỹ sư tiến hành bẻ gãy cấu trúc dữ liệu, ép kiểu trọng số từ định dạng số thực dấu phẩy động (float32) nguyên bản xuống các định dạng số nguyên 8-bit hoặc dấu phẩy tĩnh (ap_fixed) phục vụ cho vi mạch FPGA ở các giai đoạn sau.

2.3 LƯỢNG TỬ HÓA TRONG MẠNG NƠ-RON (QUANTIZATION)

2.3.1 Rào cản của dấu phẩy động

Trong các khuôn khổ huấn luyện mô hình học sâu tiêu chuẩn (như TensorFlow hay PyTorch), toàn bộ hệ thống ma trận tham số và luồng dữ liệu kích hoạt mặc định được biểu diễn dưới định dạng số thực dấu phẩy động 32-bit chuẩn IEEE 754 (float32). Xét về mặt vi kiến trúc, một véc-tơ float32 được cấu thành từ ba trường dữ liệu độc lập: 1 bit xác định dấu (sign), 8 bit biểu diễn số mũ (exponent) và 23 bit dành cho phần định trị (mantissa). Cấu trúc này mang lại dải động (dynamic range) cực kỳ rộng và độ chính xác cao, vốn là điều kiện lý tưởng cho pha huấn luyện thuật toán trên các máy chủ đám mây.

Tuy nhiên, sự phức tạp của định dạng float32 lộ rõ khi hệ thống phải thực thi các phép toán nhân cộng tích lũy (MAC) - hạt nhân tính toán nền tảng chiếm đến hơn 90% khối lượng công việc của một mạng nơ-ron. Để hoàn thành chỉ một phép tính MAC trên dữ liệu float32, kiến trúc phần cứng phải kích hoạt một chuỗi các thao tác vi lệnh cực kỳ cồng kềnh. Cụ thể, bộ xử lý phải so sánh và tính toán độ chênh lệch của các số mũ, dịch bit phần định trị của toán hạng nhỏ hơn để đồng nhất chiều không gian, thực hiện phép nhân/cộng, và cuối cùng là một chu kỳ chuẩn hóa lại kết quả (renormalization) để khớp với chuẩn IEEE 754.

Trên các nền tảng thiết bị biên (Edge Devices), sự vắng mặt của các bộ đồng xử lý toán học công suất cao (FPU) khiến các thao tác nói trên trở thành một gánh nặng lớn. Khi tiến hành tổng hợp mạch (Logic Synthesis) để đưa trực tiếp mô hình Autoencoder nguyên bản chứa các tham số float32 xuống vi mạch khả trình FPGA, hệ thống lập tức đối mặt với hiện tượng hết tài nguyên logic nghiêm trọng.

Những giới hạn tài nguyên này khẳng định rằng, việc nhúng trực tiếp một mô hình AI chưa qua tinh chỉnh xuống môi trường FPGA là điều bất khả thi, đòi

hỏi bắt buộc phải có sự can thiệp của các kỹ thuật ép kiểu dữ liệu ở tầng vi kiến trúc [6].

Đối với dòng chip nhắm tới trong đồ án là Xilinx Artix-7, tài nguyên xử lý tín hiệu số (DSP48E1) và bảng tra cứu logic (LUTs) là vô cùng hữu hạn. Việc hiện thực hóa một bộ nhân cộng float32 tiêu tốn số lượng cổng logic cao gấp hàng chục lần so với mạch toán học số nguyên thông thường. Khi hàng ngàn nút nơ-ron của mạng Autoencoder đồng loạt yêu cầu khởi tạo các khối MAC float32, phần mềm biên dịch sẽ hết tài nguyên DSP chuyên dụng và buộc phải sử dụng các LUTs đa dụng để chắp vá mạch toán học. Điều này dẫn đến lỗi nghẽn định tuyến (Routing Congestion) do lượng dây dẫn tín hiệu quá dày đặc. Hậu quả là tín hiệu truyền đi không kịp thời gian yêu cầu (timing violations) và chip không đủ diện tích để tổng hợp mạch (fitting failure).

2.3.2 Định dạng số dấu phẩy tĩnh

Để giải quyết bài toán thất cổ chai tài nguyên do định dạng dấu phẩy động gây ra, định dạng số học dấu phẩy tĩnh (Fixed-point arithmetic) được áp dụng như một giải pháp thay thế cốt lõi trong quy trình thiết kế vi mạch. Trong công cụ tổng hợp mức cao Vitis HLS của Xilinx, định dạng này được hiện thực hóa thông qua cấu trúc dữ liệu `ap_fixed<W, I>`. Theo nguyên lý tổ chức không gian nhớ, tham số `W` (Word length) định nghĩa tổng số lượng bit của toàn bộ thanh ghi, trong khi tham số `I` (Integer length) quy định số lượng bit được phân bổ riêng cho phần nguyên (đã bao hàm cả 1 bit kiểm soát dấu đối với dữ liệu có dấu). Không gian bộ nhớ còn lại, được tính bằng hiệu số ($W - I$), sẽ đảm nhận nhiệm vụ biểu diễn phần thập phân.

Điểm khác biệt then chốt của định dạng này nằm ở cơ chế vận hành tại tầng vật lý (RTL). Các mạch điện logic được tổng hợp ra từ luồng thiết kế HLS sẽ đối xử với cấu trúc `ap_fixed` tương đương như quá trình xử lý một số nguyên (Integer) thông thường. Vị trí của dấu phẩy thập phân thực chất chỉ là một ranh giới ảo (virtual radix point) do kỹ sư quy ước ở mức phần mềm để quản lý biên độ tín hiệu. Hệ thống không yêu cầu phải khởi tạo thêm bất kỳ khối logic vật lý nào để theo dõi hay dịch chuyển số mũ như ở định dạng float32. Nhờ đó, độ phức tạp của các

bộ nhân cộng (MAC) được tối giản hóa đến mức tối đa, tiết kiệm một lượng lớn công logic LUTs và giảm thiểu áp lực lên hệ thống dây dẫn định tuyến.

Trong vi kiến trúc của mạng Autoencoder được đề xuất, độ phân giải của các bộ cộng dồn (Accumulator) tại mỗi nút nơ-ron được tinh chỉnh chiến lược ở cấu hình `ap_fixed<12, 4>`. Cấu hình này cung cấp một không gian dữ liệu 12-bit tổng cộng, bao gồm 4-bit cho phần nguyên và 8-bit cho phần thập phân. Việc phân bổ 8-bit thập phân thiết lập một độ phân giải cực mịn với bước nhảy biên độ ở mức 2^{-8} (xấp xỉ 0.0039). Bước nhảy toán học này đóng vai trò then chốt trong việc bảo toàn độ nhạy của các giá trị kích hoạt cực nhỏ, tránh hiện tượng "chết nơ-ron" (neuron death) do tín hiệu bị làm tròn cơ học về không. Đồng thời, không gian 4-bit phần nguyên cung cấp một vùng đệm an toàn để chống lại hiện tượng tràn số vật lý (overflow) khi hàng chục tín hiệu điểm ảnh ngõ vào được nhân cộng và tích lũy đồng loạt tại một điểm nút duy nhất.

2.3.3 Phương pháp Lượng tử hóa nhận thức (QAT) và thư viện QKeras

Quá trình chuyển đổi ép kiểu dữ liệu từ không gian số thực liên tục sang số hữu hạn luôn đi kèm với rủi ro suy hao toán học nghiêm trọng. Nếu áp dụng phương pháp Lượng tử hóa sau huấn luyện (Post-Training Quantization - PTQ) truyền thống - tức là tiến hành huấn luyện xong mô hình bằng cấu trúc float32 đạt độ chính xác tối ưu rồi mới thực hiện cắt gọt bit cơ học - sự phân bố trọng số của mạng sẽ bị phá vỡ đột biến (quantization shock). Đối với đặc thù của bài toán phục dựng tín hiệu thị giác (như mạng Autoencoder), phương pháp PTQ gây ra sự mất mát thông tin không thể phục hồi, làm biến dạng các đường nét hình học cốt lõi và tạo ra hiệu ứng nhiễu hạt lớn trên ma trận điểm ảnh ngõ ra.

Để vượt qua rào cản toán học này, QAT sử dụng kỹ thuật ước lượng Straight-Through Estimator (STE) trong pha lan truyền ngược (backward pass) [7]. STE đóng vai trò như một cầu nối giả lập, cho phép gradient đi xuyên qua các lớp lượng tử hóa không khả vi, từ đó liên tục cập nhật ma trận trọng số và hướng mạng tiệm cận đến một điểm hội tụ mới nhằm tự động bù đắp các sai số vật lý.

Để giải quyết rào cản này, đề án áp dụng kỹ thuật Lượng tử hóa nhận thức (QAT) tiên tiến. Cơ chế hoạt động cốt lõi của thuật toán QAT dựa trên việc chủ động cấy ghép các nhiễu làm tròn (quantization noise) vào pha lan truyền xuôi

(forward pass) ngay trong quá trình mạng nơ-ron đang học. Sự can thiệp này mô phỏng chính xác hành vi cắt gọt số thập phân sẽ diễn ra trên phần cứng vật lý. Tuy nhiên, do các hàm bậc thang làm tròn không có đạo hàm liên tục, thuật toán tối ưu hóa sẽ bị triệt tiêu gradient.

Sự đồng bộ từ trên xuống dưới (top-down) này đảm bảo mô hình Tự mã hóa sau khi dịch mã C++ qua hls4ml cho kết quả C-Simulation nhất quán với mô hình Python gốc, xác nhận quá trình lượng tử hóa không gây sai lệch logic.

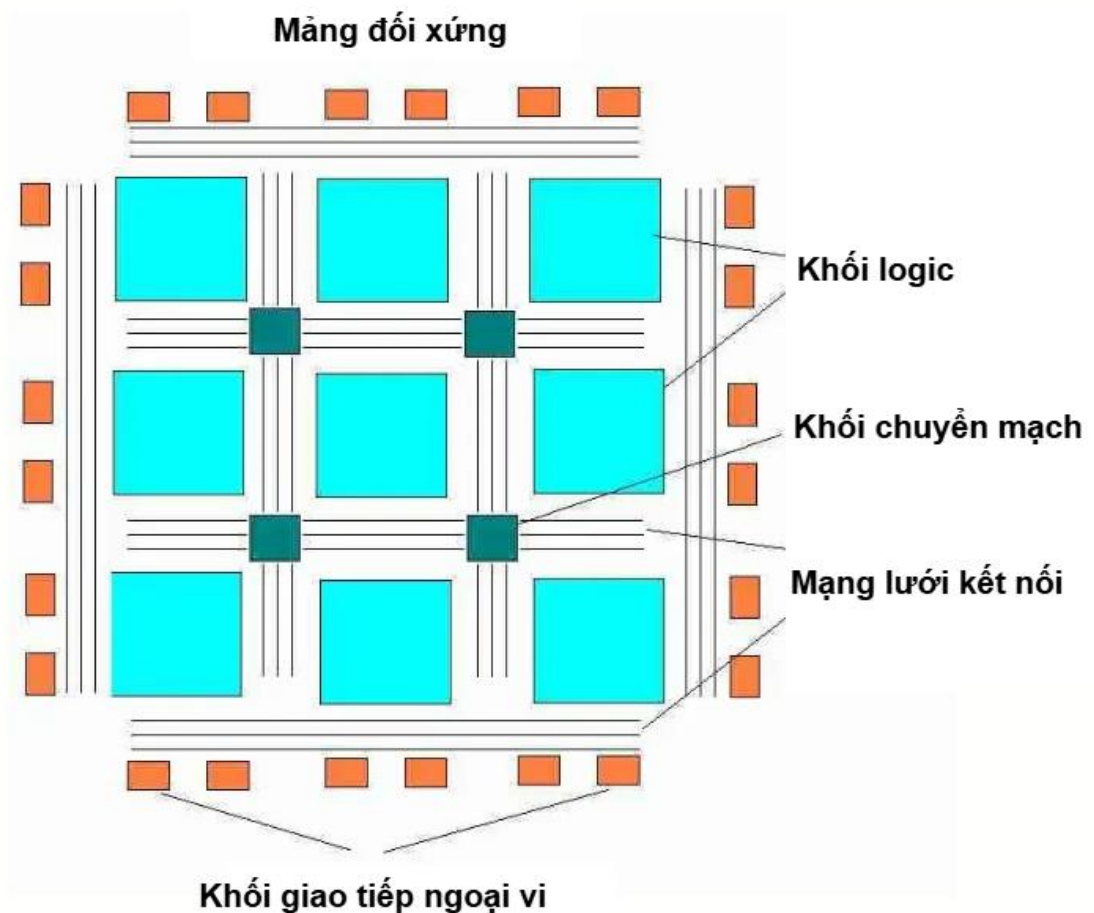
Nhằm thực thi cơ chế QAT một cách đồng bộ nhất với phần cứng, nghiên cứu đã khai thác sức mạnh của thư viện mã nguồn mở QKeras [8]. Trong không gian mô phỏng học sâu, các lớp nơ-ron tiêu chuẩn được thay thế bằng cấu hình lớp lượng tử hóa QDense, kết hợp với hàm định dạng `quantized_bits(8, 3, 1)`. Tham số này thiết lập chính xác một không gian dữ liệu 8-bit, bao gồm 3-bit cho phần nguyên và 1-bit xác định dấu (tương đương với 4-bit không gian nguyên). Việc áp đặt định dạng toán học nghiêm ngặt này tạo ra một sự nhất quán giữa không gian thuật toán phần mềm trên Python và vi kiến trúc mạch điện trên môi trường tổng hợp C++ (Vitis HLS).

2.4 NỀN TẢNG VI MẠCH KHẢ TRÌNH FPGA VÀ DÒNG XILINX ARTIX-7

2.4.1 Tổng quan về FPGA

Vi mạch khả trình hiện trường (FPGA) là một nền tảng bán dẫn cho phép tái cấu trúc phần cứng ở mức logic ngay cả sau khi đã chế tạo. Khác với kiến trúc von Neumann trên CPU hay GPU - nơi các lệnh phần mềm được nạp và thực thi tuần tự qua một đường dẫn dữ liệu cố định - FPGA hoạt động theo cơ chế song song không gian (spatial parallelism). Nhờ đó, kỹ sư có thể ánh xạ trực tiếp các thuật toán học sâu thành những luồng dữ liệu (dataflow) chuyên biệt và các mạch

chạy đồng thời trên bề mặt lõi silicon.



Hình 2.2 Sơ đồ vi kiến trúc tổng quát của nền tảng vi mạch FPGA.

Hình 2.2 thể hiện, một cấu trúc FPGA tiêu chuẩn gồm các khối logic sắp xếp thành mảng đối xứng, liên kết với nhau qua mạng lưới kết nối và các khối chuyển mạch, cùng các khối giao tiếp ngoại vi ở rìa chip. Xét về tài nguyên vật lý, thiết kế dựa trên ba thành phần chính: khối logic khả trình, khối bộ nhớ tĩnh cục bộ và khối xử lý tín hiệu số chuyên dụng.

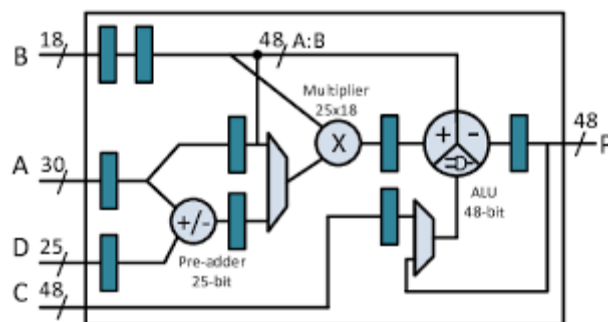
Khối logic Configurable Logic Block – CLB) gồm các bảng tra cứu logic (Look-Up Table – LUT) và thanh ghi (Flip-Flop), là nền tảng để tổng hợp các hàm Boolean, mạch tổ hợp và máy trạng thái tuần tự [9]. Khối bộ nhớ tĩnh cục bộ (Block RAM – BRAM) được phân bố rải rác trên toàn đế chip, cung cấp không gian lưu trữ và băng thông truy xuất ngay trên chip (on-chip memory), giúp giảm độ trễ thất cổ chai so với truy xuất từ bộ nhớ ngoài. Khối xử lý tín hiệu số chuyên dụng (DSP Slice) là các lõi cứng tích hợp sẵn từ mạch nhân cộng tối ưu ở tầng vật lý, phục vụ

trực tiếp cho các tác vụ toán học dày đặc như phép nhân ma trận trong mạng nơ-ron.

2.4.2 Kiến trúc tài nguyên chip Artix-7 100T

Nền tảng phần cứng trung tâm để thực thi mô hình Tự mã hóa trong đồ án là vi mạch khả trình Xilinx Artix-7, mã linh kiện xc7a100tcsq324-1. Thuộc phân khúc thiết bị biên tối ưu về chi phí và công suất trên tiến trình 28 nm, dòng chip này có giới hạn tài nguyên chặt: 63.400 bảng tra cứu logic (Look-Up Table – LUT) để xây dựng hàm tổ hợp, 126.800 thanh ghi trạng thái (Flip-Flop – FF) cho lưu trữ tuần tự, 135 khối bộ nhớ tĩnh cục bộ (Block RAM – BRAM) để lưu tham số nội bộ, và 240 khối xử lý tín hiệu số chuyên dụng DSP48E1. Giới hạn tài nguyên này đặt ra một rào cản, đòi hỏi mạng nơ-ron phải được tinh chỉnh và lượng tử hóa để tránh tràn tài nguyên tổng hợp và nghẽn định tuyến.

Trong số đó, 240 khối DSP48E1 là tài nguyên quan trọng nhất, quyết định năng lực tính toán của bộ gia tốc. Mỗi khối DSP48E1 là một lõi cứng (hard IP block) tích hợp sẵn trên silicon, gồm một bộ nhân 25×18 bit kết nối trực tiếp với bộ cộng/tích lũy (Accumulator) 48-bit và một đơn vị logic số học (ALU) [10]. Cấu trúc này phản ánh đúng phép nhân cộng tích lũy (MAC) - thành phần chiếm khối lượng tính toán lớn nhất trong mạng Autoencoder.



Hình 2.3 Sơ đồ vi kiến trúc chi tiết của lõi xử lý tín hiệu số DSP48E1 trên FPGA Xilinx 7-Series.

Sơ đồ trong Hình 2.3 cho thấy đường dữ liệu của một khối DSP48E1: hai ngõ vào đi qua bộ tiền cộng (pre-adder 25-bit), vào bộ nhân 25×18 , rồi tới ALU

48-bit để cộng dồn, cho ngõ ra P rộng 48-bit. Đây chính là cấu trúc phần cứng tương ứng với một phép MAC.

Việc khai thác các khối cứng này có ý nghĩa quyết định với đồ án. Nếu thực hiện phép nhân ma trận bằng LUT thay vì DSP, thiết kế sẽ tốn quá nhiều tài nguyên logic và đường định tuyến, làm giảm tần số tối đa và có thể khiến quá trình tổng hợp thất bại (timing failure). Ngược lại, ánh xạ trực tiếp các phép MAC đã lượng tử hóa xuống 240 khối DSP48E1 cho phép kiến trúc gia tốc song song trên diện rộng: hệ thống có thể thực thi hàng trăm chuỗi nhân ma trận song song trong cùng một chu kỳ xung nhịp, tăng thông lượng (throughput) và giảm độ trễ suy luận, đáp ứng yêu cầu của hệ thống Edge AI.

2.5 CÔNG CỤ CHUYỂN ĐỔI HLS VÀ THƯ VIỆN HLS4ML

2.5.1 Khái niệm High-Level Synthesis (HLS)

High-Level Synthesis (HLS) là phương pháp thiết kế phần cứng cho phép tự động chuyển đổi các mô tả thuật toán ở ngôn ngữ lập trình bậc cao như C, C++ hoặc SystemC thành mã mô tả phần cứng ở mức thanh ghi truyền dữ liệu (Register Transfer Level – RTL) [11]. Thông qua HLS, các nhà phát triển có thể tập trung vào việc xây dựng và tối ưu thuật toán mà không cần thiết kế trực tiếp bằng các ngôn ngữ mô tả phần cứng như VHDL hoặc Verilog.

Quá trình tổng hợp HLS thường bao gồm ba giai đoạn chính: lập lịch (scheduling), phân bổ tài nguyên (allocation) và gắn kết tài nguyên (binding). Trong giai đoạn lập lịch, các phép toán được sắp xếp vào các chu kỳ xung nhịp phù hợp nhằm đáp ứng các ràng buộc về hiệu năng và thời gian. Tiếp theo, giai đoạn phân bổ tài nguyên xác định số lượng và loại phần tử phần cứng cần sử dụng, chẳng hạn như bộ cộng, bộ nhân hoặc bộ nhớ. Cuối cùng, giai đoạn gắn kết tài nguyên thực hiện việc ánh xạ các phép toán và dữ liệu vào các tài nguyên phần cứng đã được phân bổ, từ đó tạo ra kiến trúc RTL hoàn chỉnh.

Nhờ khả năng rút ngắn thời gian phát triển và nâng cao năng suất thiết kế, HLS ngày càng được ứng dụng rộng rãi trong các lĩnh vực xử lý tín hiệu số, trí tuệ nhân tạo và thiết kế hệ thống nhúng trên FPGA.

2.5.2 Cơ chế biên dịch đồ thị tính toán của thư viện hls4ml

Trong luồng thiết kế đồng bộ từ mô phỏng phần mềm đến hiện thực hóa phần cứng biên, thư viện hls4ml là công cụ trung gian chuyển đổi mô hình học sâu thành kiến trúc mạch điện vật lý [12]. Cơ chế vận hành của hls4ml bắt đầu bằng việc tiếp nhận và bóc tách toàn diện cấu trúc đồ thị tính toán (computation graph) từ tập tin định dạng .h5 – cấu trúc lưu trữ tiêu chuẩn chứa thông tin về cấu hình các lớp nơ-ron, chuỗi liên kết hàm kích hoạt, cũng như toàn bộ ma trận trọng số và véc-tơ độ lệch đã trải qua huấn luyện. Thư viện tiến hành phân tích cú pháp đồ thị này, phân rã các toán tử phức tạp và ánh xạ từng nút tính toán của mạng Autoencoder sang các cấu trúc phần tử phần cứng tương đương được định nghĩa sẵn trong hệ thống thư viện mẫu C++ của hls4ml, đảm bảo tính toàn vẹn và chính xác của cấu trúc liên kết ban đầu.

Sau khi hoàn thành pha bóc tách và diễn dịch đồ thị, hls4ml tự động thực thi quy trình sinh mã nguồn C++ có độ tương thích với công cụ tổng hợp mức cao Vitis HLS [13]. Thay vì yêu cầu kỹ sư phải thiết kế thủ công các mạch nhân chập hay các hàm phi tuyến phức tạp ở mức thanh ghi, hls4ml tự động tối ưu hóa cấu trúc mã nguồn bằng cách tích hợp sẵn các hàm định kiểu dữ liệu của Xilinx. Đặc biệt, toàn bộ các tham số trọng số và độ lệch sau khi trích xuất sẽ được hls4ml đóng gói một cách tự động thành các mảng hằng số tĩnh (static arrays) dưới định dạng số dấu phẩy tĩnh `ap_fixed` chuyên biệt. Các mảng hằng số này được cấu trúc và tổ chức lưu trữ tối ưu hóa về mặt bố cục bộ nhớ, sẵn sàng để trình biên dịch ánh xạ trực tiếp và phân phối lên các khối bộ nhớ cục bộ BRAM của chip Artix-7. Việc tự động hóa chu trình chuyển đổi này loại bỏ các sai sót định kiểu thủ công, thiết lập tính nhất quán toán học giữa pha mô phỏng thuật toán trên Python và pha thực thi mạch điện RTL ở các giai đoạn sau.

2.5.3 Các chiến lược tối ưu hóa kiến trúc vi mạch

Mặc dù chiến lược này tối đa hóa tính song song không gian và mang lại độ trễ thấp nhất, nó lại tiêu tốn một lượng tài nguyên logic rất lớn, dễ dàng dẫn đến rủi ro hết 240 khối DSP48E1 hữu hạn trên dòng chip Artix-7. Ngược lại, khi cấu hình hệ số tái sử dụng ở mức cao, hệ thống sẽ áp dụng kỹ thuật ghép kênh thời gian (time-multiplexing). Cơ chế này cho phép một khối DSP vật lý duy nhất được

tái sử dụng tuần tự qua nhiều chu kỳ xung nhịp để tính toán cho nhiều luồng dữ liệu khác nhau, giúp tiết kiệm tối đa diện tích lõi silicon và giải quyết bài toán thất cổ chai tài nguyên, nhưng bù lại sẽ làm tăng tổng thời gian thực thi của mô hình.

Trong khuôn khổ của thư viện hls4ml và công cụ tổng hợp Vitis HLS, quá trình tối ưu hóa kiến trúc vi mạch không chỉ dừng lại ở kỹ thuật lượng tử hóa dữ liệu mà còn phụ thuộc sâu sắc vào chiến lược phân bổ tài nguyên không gian và thời gian. Nòng cốt của chiến lược này được định hình và kiểm soát chặt chẽ thông qua cấu hình Hệ số tái sử dụng (Reuse Factor). Đây là một cơ chế định tuyến vi kiến trúc cho phép kỹ sư cân bằng trực tiếp giữa diện tích phần cứng tiêu thụ và độ trễ suy luận (latency) của mạng nơ-ron. Khi hệ số tái sử dụng được thiết lập ở giá trị tối thiểu (Reuse Factor bằng 1), bộ biên dịch sẽ thực thi chiến lược khai triển phẳng (fully unrolled), khởi tạo độc lập hàng loạt các khối nhân cộng DSP cho từng phép tính MAC bên trong các lớp QDense.

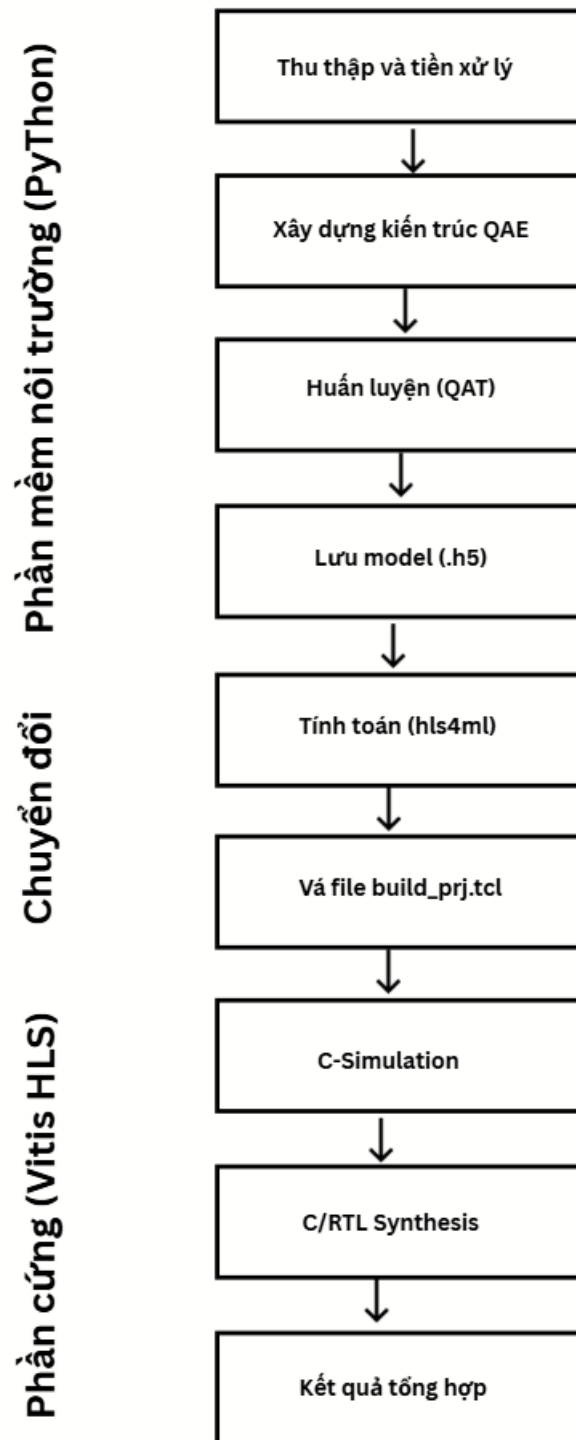
Sự phối hợp đồng bộ giữa cơ chế ghép kênh tài nguyên DSP và chiến lược mở rộng băng thông BRAM đã tạo ra một kiến trúc phần cứng tổng thể tinh gọn, vừa thỏa mãn các ràng buộc khắt khe về diện tích vật lý của nền tảng thiết bị biên, vừa duy trì được tốc độ suy luận thời gian thực cho mạng Tự mã hóa.

Bên cạnh kỹ thuật điều tiết khối xử lý toán học, hiệu năng của vi kiến trúc còn được tinh chỉnh sâu thông qua các chiến lược đường ống hóa (Pipelining) và phân mảnh không gian bộ nhớ (Array Partitioning) được hls4ml tích hợp tự động. Bằng cách thiết lập chiến lược tối ưu hóa độ trễ (Latency Strategy) hoặc thông lượng (Resource Strategy) trong tệp cấu hình, hls4ml sẽ sinh ra các chỉ thị mã hóa tương ứng để điều phối luồng dữ liệu. Kiến trúc đường ống cho phép các tầng nơ-ron tính toán gối đầu liên tục; ngay khi một phần ma trận dữ liệu vừa rời khỏi lớp mã hóa, nó lập tức được đưa vào lớp thất cổ chai xử lý mà không cần chờ toàn bộ bức ảnh hoàn tất. Để đáp ứng băng thông cấp liệu rất lớn cho các đường ống song song này, ma trận trọng số và các véc-tơ kích hoạt được phân mảnh một cách có chủ đích, bẻ nhỏ các khối bộ nhớ BRAM thành nhiều bank nhớ nội bộ có khả năng đọc/ghi đồng thời.

CHƯƠNG 3: THIẾT KẾ VÀ TỐI ƯU HÓA HỆ THỐNG

3.1 SƠ ĐỒ QUY TRÌNH THỰC HIỆN

Quy trình thực hiện tổng thể của hệ thống được xây dựng dựa trên nền tảng phương pháp luận Đồng thiết kế phần cứng và phần mềm (Hardware/Software Co-design). Phương pháp này cho phép khai thác tối đa sức mạnh mô hình hóa của các thư viện học sâu cấp cao, đồng thời tận dụng năng lực tính toán song song ở cấp độ cổng logic. Trục thời gian của dự án được chia thành ba giai đoạn phân cấp tiếp nối nhau một cách chặt chẽ, đảm bảo đồ thị toán học được dịch chuyển chính xác từ không gian số thực có độ chính xác cao sang cấu trúc vi mạch vật lý tối ưu năng lượng. Sơ đồ quy trình dưới đây phác họa toàn cảnh luồng xử lý tín hiệu đi qua ba môi trường cốt lõi bao gồm: lớp phần mềm mô phỏng, lớp chuyển đổi trung gian và lớp tổng hợp phần cứng.



Hình 3.1 Sơ đồ quy trình thực hiện đồ án theo phương pháp đồng thiết kế phần cứng - phần mềm.

Giai đoạn đầu tiên của hệ thống tập trung vào việc định hình thuật toán trên môi trường phần mềm Python. Khởi đầu chu trình, tập dữ liệu ảnh chữ số viết tay

MNIST được thu thập và phải trải qua một quy trình tiền xử lý nghiêm ngặt nhằm giảm tải áp lực tài nguyên cho phần cứng sau này. Dữ liệu được phân bổ thành 60.000 mẫu huấn luyện và 10.000 mẫu kiểm thử, đồng thời biên độ điểm ảnh được chuẩn hóa về dải $[0, 1]$. Đặc biệt, một lớp lấy mẫu cực đại (MaxPool) được áp dụng để nén không gian ảnh từ 28×28 xuống còn 14×14 , giúp giảm mạnh số lượng điểm ảnh ngõ vào xuống mức 196. Dựa trên cơ sở dữ liệu này, kiến trúc mạng Autoencoder lượng tử hóa (QAE) được xây dựng thông qua thư viện QKeras. Đồ thị mạng được thiết kế theo cấu trúc thắt nút cổ chai (bottleneck) 196-16-8-16-196, tích hợp định dạng lượng tử hóa 8-bit trên tất cả các lớp QDense. Để bù đắp lượng thông tin suy hao do quá trình ép kiểu dữ liệu, hệ thống tiến hành quá trình Huấn luyện nhận thức lượng tử (QAT) trong 50 chu kỳ kết hợp cùng bộ tối ưu hóa Adam. Khi hàm suy hao trung bình bình phương (MSE) trên tập xác thực (validation) hội tụ ở mức 0.032, mạng đã học được các đặc trưng cốt lõi. Toàn bộ 6.764 tham số lúc này được trích xuất và đóng gói tĩnh thành tệp định dạng .h5, đạt dung lượng siêu nhỏ ở mức xấp xỉ 6.6 KB, tạo tiền đề lý tưởng cho việc nhúng lên chip.

Bước sang giai đoạn thứ hai, hệ thống thiết lập một lớp cầu nối chuyên đổi để vượt qua rào cản ngôn ngữ lập trình. Công cụ mã nguồn mở hls4ml được huy động để biên dịch tự động đồ thị toán học từ không gian Keras sang ngôn ngữ C++ có khả năng tổng hợp vi mạch. Quá trình ánh xạ này được cấu hình nhằm mục tiêu cực kỳ chi tiết, chỉ định rõ mã linh kiện FPGA Artix-7 (xc7a100tcs324-1), cài đặt chu kỳ xung nhịp 10 ns (tương đương tần số hoạt động 100 MHz) và áp dụng giao thức truyền luồng dữ liệu nối tiếp. Một thao tác can thiệp kỹ thuật mang tính bắt buộc trong giai đoạn này là bước tinh chỉnh tệp mã lệnh hệ thống cấu hình. Lệnh phân mảnh mảng dữ liệu mặc định được vô hiệu hóa thông qua việc vá file báo cáo cấu hình, nhằm tránh các rủi ro xung đột biên dịch khi giao tiếp với môi trường Xilinx Vivado phiên bản 2022.2.

Giai đoạn cuối cùng là khâu quyết định, được thực thi trực tiếp trên nền tảng Xilinx Vitis HLS để hiện thực hóa lỗi gia tốc vật lý. Trước khi hệ thống khởi động trình tổng hợp, mã nguồn C++ bắt buộc phải trải qua khâu kiểm chứng mô phỏng hành vi (C-Simulation) nhằm xác nhận tính tương đương bit (bit-accurate). Việc hệ thống ghi nhận không xảy ra bất kỳ lỗi sai số toán học nào khi đối chiếu

kết quả ngõ ra với môi trường Python là minh chứng khẳng định quá trình lượng tử hóa đã thành công. Vượt qua khâu kiểm chứng, tiến trình Tổng hợp vi mạch mức RTL chính thức được khởi động. Tại đây, cấu trúc định tuyến luồng dữ liệu (Dataflow) được áp dụng để dàn phẳng và tạo cơ chế gói đầu song song cho các phép toán. Kết quả trích xuất cuối cùng chính thức đánh dấu sự hoàn thiện của bộ gia tốc vi mạch với thông số độ trễ suy luận chỉ 91 chu kỳ. Báo cáo cấu hình phần cứng cũng xác nhận việc khai thác hợp lý quỹ tài nguyên Artix-7 với mức tiêu thụ lỗi xử lý tín hiệu số DSP đạt 50.00% và bộ nhớ BRAM đạt 47.41%. Các chỉ số này mang lại một hiệu năng với thông lượng xử lý đạt 1.56 triệu khung hình/giây, sức mạnh tính toán 20.40 GOP/s, trong khi vẫn duy trì công suất tiêu thụ lý tưởng ở mức 0.665 W.

3.2 THIẾT KẾ MÔ HÌNH PHẦN MỀM

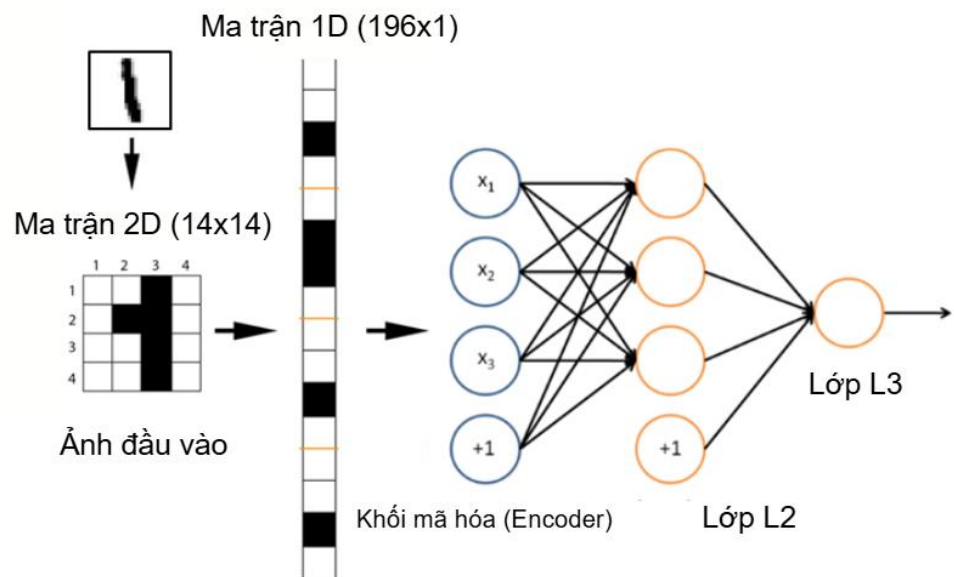
3.2.1 Thu thập và tiền xử lý tập dữ liệu ảnh MNIST

Tập dữ liệu MNIST (Modified National Institute of Standards and Technology) gồm 70.000 ảnh chữ số viết tay từ 0 đến 9, được dùng phổ biến làm chuẩn đối sánh trong thị giác máy tính và học máy siêu nhỏ (TinyML). Quy trình bắt đầu bằng việc trích xuất tập dữ liệu từ thư viện mẫu của TensorFlow/Keras và chia thành hai tập độc lập: 60.000 ảnh cho huấn luyện và 10.000 ảnh cho kiểm thử. Mỗi ảnh gốc là một ma trận hai chiều kích thước 28×28 điểm ảnh, trong đó cường độ mỗi điểm ảnh là một số nguyên từ 0 đến 255 theo thang độ xám, từ nền đen đến nét vẽ trắng.

Để tăng tốc độ hội tụ của mạng và tránh hiện tượng bùng nổ gradient khi lan truyền ngược, dữ liệu được chuẩn hóa biên độ điểm ảnh. Phép chuẩn hóa thực hiện bằng cách chia cường độ mỗi điểm ảnh cho giá trị cực đại 255, đưa toàn bộ dữ liệu từ số nguyên sang số thực dấu phẩy động trong khoảng $[0, 1]$. Việc đưa dữ liệu về cùng một dải giá trị giúp các hàm kích hoạt phi tuyến ở các tầng ẩn hoạt động ổn định hơn, đồng thời tạo điều kiện thuận lợi cho bước lượng tử hóa nhận thức (QAT) trên QKeras ở các bước sau.

Sau khi chuẩn hóa, ảnh được giảm kích thước bằng MaxPooling để giảm khối lượng tính toán và tiết kiệm BRAM cho FPGA. Một bộ lọc MaxPooling của

số 2×2 với bước dịch (stride) bằng 2 được quét tuần tự, không chồng lấp, trên toàn bộ ảnh 28×28 . Tại mỗi vùng cửa sổ đi qua, hệ thống giữ lại giá trị điểm ảnh lớn nhất và loại bỏ các pixel nhiễu nền hoặc dư thừa quanh nét vẽ. Phép toán này giảm kích thước ảnh từ 28×28 xuống 14×14 pixel. Cuối cùng, ma trận 14×14 được trải phẳng (flatten) thành một mảng một chiều gồm 196 phần tử. Mảng 196 phần tử này chính là véc-tơ ngõ vào tiêu chuẩn, sẵn sàng nạp vào các tầng nơ-ron của bộ mã hóa (Encoder), như minh họa trong Hình 3.2.



Hình 3.2 Sơ đồ biểu diễn phép biến đổi ma trận 2D thành véc-tơ 1D cấp cho lớp nơ-ron của mạng Autoencoder.

3.2.2 Đề xuất kiến trúc mạng Autoencoder tinh gọn

Để có thể triển khai thành công mô hình học sâu lên hệ thống phần cứng thiết bị biên có tài nguyên hữu hạn như vi mạch FPGA Artix-7, kiến trúc của mạng nơ-ron phải được tối ưu hóa ngay từ pha thiết kế thuật toán phần mềm. Thay vì sử dụng các mạng Tích chập phức tạp với hàng triệu tham số, nghiên cứu này đề xuất một cấu trúc mạng Tự mã hóa (Autoencoder) cực kỳ tinh gọn, tuân thủ nghiêm ngặt nguyên lý thiết kế hình học đối xứng qua gương (mirror-like symmetry). Cấu trúc vi mô của mạng được thiết lập thành một chuỗi phân lớp giảm và tăng chiều dữ liệu mạnh, với số lượng nút nơ-ron tại mỗi tầng lần lượt là 196, 16, 8, 16 và 196.

Tại khối Mã hóa (Encoder), véc-tơ dữ liệu ngõ vào phẳng gồm 196 phần tử (tương ứng với bức ảnh đã qua xử lý Max Pooling) được kết nối trực tiếp với một tầng ẩn đầu tiên chứa 16 nút nơ-ron. Sự thu hẹp không gian dữ liệu một cách đột ngột này ép buộc thuật toán phải chắt lọc, trích xuất và chỉ giữ lại những đặc trưng hình học cốt lõi nhất của nét chữ viết tay, đồng thời loại bỏ các điểm ảnh nhiễu nền không mang giá trị thông tin. Tiếp nối sau đó, luồng dữ liệu bị nén tới hạn vào tầng thắt cổ chai (bottleneck) hay còn gọi là không gian tiềm ẩn (latent space), được cấu thành từ đúng 8 nút nơ-ron. Đây là trạng thái biểu diễn thông tin ở mức cô đặc và trừu tượng nhất của toàn bộ hệ thống đồ thị tính toán.

Từ lõi thắt cổ chai này, khối Giải mã (Decoder) làm nhiệm vụ tiếp nhận véc-tơ 8 phần tử và tiến hành pha phục dựng tín hiệu thị giác theo cấu trúc phản xạ ngược lại với khối Mã hóa. Thông tin được giải nén và nội suy qua một tầng ẩn 16 nơ-ron trước khi khuếch tán ra tầng ngõ ra cuối cùng chứa 196 nơ-ron, nhằm tái tạo lại nguyên bản ma trận ảnh 14x14 ban đầu. Việc thiết lập chuỗi nơ-ron phân lớp nghiêm ngặt (196 -> 16 -> 8 -> 16 -> 196) đóng vai trò then chốt trong việc giảm thiểu tối đa số lượng tham số trọng số (weights) và độ lệch (biases) cần huấn luyện. Thiết kế vi mô tinh gọn này không chỉ giúp định tuyến luồng dữ liệu rõ ràng, ngăn chặn hiện tượng quá khớp (overfitting) trên môi trường phần mềm, mà còn tạo ra một tiền đề vật lý: đảm bảo toàn bộ bộ tham số của mô hình có kích thước đủ nhỏ để được lưu trữ bên trong các khối bộ nhớ tĩnh cục bộ (BRAM) của lõi chip Artix-7, từ đó giảm sự phụ thuộc và độ trễ truy xuất từ các hệ thống bộ nhớ ngoại vi.

3.2.3 Cấu hình tham số lượng tử hóa với QKeras

Để hiện thực hóa phương pháp Lượng tử hóa nhận thức (QAT) đã được đề cập, toàn bộ các tầng nơ-ron kết nối đầy đủ (Dense layers) tiêu chuẩn trong không gian Keras được thay thế bằng cấu trúc lớp QDense của thư viện QKeras. Tại mỗi lớp QDense, hạt nhân tính toán được định hình và kiểm soát nghiêm ngặt bởi hàm định dạng `quantized_bits(8, 3, 1)`. Về mặt vi kiến trúc số, cấu hình này ép buộc toàn bộ ma trận trọng số (weights) và véc-tơ độ lệch (biases) của mạng phải được biểu diễn dưới định dạng số học dấu phẩy tĩnh 8-bit. Trong không gian thanh ghi này, dữ liệu được phân bổ theo một tỷ lệ chiến lược: 3-bit dành cho phần nguyên,

1-bit kiểm soát dấu, và 4-bit còn lại đảm nhận độ phân giải thập phân. Không gian 8-bit tổng thể này là một điểm cân bằng lý tưởng, vừa đủ độ phân giải toán học để mạng Autoencoder giữ lại năng lực phục dựng hình ảnh, vừa khớp với chiều rộng băng thông của các bộ nhân nguyên bản bên trong lõi DSP48E1 trên nền tảng Artix-7, giúp tránh lãng phí tài nguyên silicon.

Đồng bộ với quá trình lượng tử hóa trọng số, cơ chế truyền dẫn tín hiệu phi tuyến qua các tầng nơ-ron cũng bị can thiệp sâu sắc. Hàm kích hoạt chuẩn ReLU được thay thế bằng hàm ReLU lượng tử hóa (quantized ReLU) có cùng cấu hình bit. Nếu giữ nguyên hàm ReLU truyền thống của TensorFlow, tín hiệu ngõ ra của các tầng nơ-ron sẽ tự động bị ép kiểu ngược về định dạng dấu phẩy động 32-bit (float32) ở pha lan truyền xuôi, phá vỡ toàn bộ quy trình thiết kế phần cứng và gây hết tài nguyên logic khi tổng hợp vi mạch. Việc áp đặt hàm kích hoạt lượng tử hóa đảm bảo rằng luồng dữ liệu kích hoạt (activations) truyền từ không gian tiềm ẩn này sang không gian tiềm ẩn khác luôn được cắt gọt, kẹp biên (clipping) và đóng gói cố định trong giới hạn 8-bit. Sự phối hợp khép kín và đồng bộ giữa lớp QDense và hàm kích hoạt lượng tử hóa tạo ra một đồ thị tính toán nhất quán về mặt kiểu dữ liệu, đóng vai trò như một nguyên mẫu tham chiếu (golden model) trên Python trước khi hệ thống được bàn giao cho hls4ml để biên dịch sang mã nguồn C++.

3.2.4 Tính toán độ phức tạp mô hình và kích thước bộ nhớ lưu trữ

Việc đánh giá độ phức tạp thuật toán và dự toán chính xác dung lượng bộ nhớ vật lý là một bước kiểm định bắt buộc trước khi tiến hành tổng hợp vi mạch. Độ phức tạp của mạng Autoencoder kết nối đầy đủ (Fully Connected) được định lượng trực tiếp thông qua tổng số lượng tham số học (trainable parameters), bao gồm ma trận trọng số (weights) và véc-tơ độ lệch (biases). Công thức toán học tổng quát để tính toán số lượng tham số P tại một lớp nơ-ron Dense bất kỳ được biểu diễn như sau:

$$P = (N_{in} \times N_{out}) + N_{out} \quad (3.1)$$

Trong đó, N_{in} là số lượng nút nơ-ron ở lớp ngõ vào và N_{out} là số lượng nút nơ-ron ở lớp ngõ ra của tầng tính toán đó. Áp dụng công thức này vào chuỗi kiến trúc đối xứng (196 -> 16 -> 8 -> 16 -> 196) đã đề xuất, ta tiến hành phân rã không gian tham số theo từng tầng vi mô. Tại khối Mã hóa, tầng ẩn đầu tiên kết nối ma trận

ngõ vào 196 phần tử với 16 nơ-ron, tạo ra một khối lượng tính toán gồm $(196 \times 16) + 16 = 3152$ tham số. Tiếp theo, tầng thất cổ chai nén dữ liệu từ 16 nơ-ron xuống 8 nơ-ron, yêu cầu bổ sung thêm $(16 \times 8) + 8 = 136$ tham số. Phản xạ qua gương tại khối Giải mã, tầng mở rộng không gian từ 8 nơ-ron lên 16 nơ-ron tiêu tốn $(8 \times 16) + 16 = 144$ tham số. Cuối cùng, tầng ngõ ra chịu trách nhiệm phục dựng bức ảnh phân rã tín hiệu từ 16 nơ-ron ra 196 nơ-ron, đòi hỏi một hệ thống $(16 \times 196) + 196 = 3332$ tham số. Tổng hợp toàn bộ đồ thị tính toán, mạng Autoencoder sở hữu tổng cộng 6764 tham số. So với các mô hình thị giác máy tính kinh điển chứa hàng triệu trọng số, đây là một con số cực kỳ khiêm tốn và tinh gọn.

Giá trị cốt lõi của phương pháp thiết kế phần cứng đồng bộ (HW-aware design) được thể hiện rõ nét nhất ở bước quy đổi từ số lượng tham số sang dung lượng bộ nhớ vật lý. Nhờ việc áp dụng kỹ thuật Lượng tử hóa nhận thức (QAT) với cấu hình hàm `quantized_bits(8, 3, 1)`, mỗi tham số đơn lẻ trong mạng (bất kể là trọng số hay độ lệch) đều được ép kiểu và biểu diễn bằng một thanh ghi dấu phẩy thập phân có kích thước chính xác là 8-bit (tương đương 1 Byte). Do đó, tổng không gian lưu trữ lý thuyết mà mô hình yêu cầu trên phần cứng chỉ là 6764 Bytes, tức xấp xỉ 6.6 KB.

Đối chiếu với giới hạn kiến trúc tài nguyên của dòng chip Xilinx Artix-7 100T - nền tảng vốn được trang bị 135 khối bộ nhớ tĩnh nội bộ (Block RAM), cung cấp tổng dung lượng lưu trữ trên chip (on-chip memory) vào khoảng 600 KB - yêu cầu 6.6 KB của mạng Autoencoder chỉ chiếm chưa tới 1.5% tổng ngân sách BRAM. Sự dư dả về mặt không gian này xác nhận tính khả thi vật lý của đề án: toàn bộ ma trận trọng số của mạng có thể được nhúng thẳng vào cấu trúc BRAM cục bộ sát cạnh các khối xử lý DSP48E1. Thiết kế này loại bỏ các chu kỳ xung nhịp trễ do phải giao tiếp truy xuất dữ liệu từ các bộ nhớ SDRAM ngoại vi, thiết lập nền tảng để tối đa hóa thông lượng suy luận thời gian thực cho bộ gia tốc.

3.3 TỔNG HỢP PHẦN CỨNG VÀ TỐI ƯU HÓA VI KIẾN TRÚC MỨC RTL

3.3.1 Quy trình dịch mã C++ tự động hóa bằng hls4ml

Bước sang giai đoạn hiện thực hóa phần cứng, quy trình thiết kế chuyển dịch toàn diện từ không gian mô phỏng thuật toán sang pha tổng hợp vi kiến trúc mức chuyển thành ghi (RTL). Trung tâm của chu trình tự động hóa này là việc khai thác thư viện hls4ml để dịch chuyển mô hình học sâu từ mã nguồn Python sang dự án C++ tương thích với môi trường Vitis HLS. Toàn bộ mã lệnh biên dịch, cấu trúc phần cứng và các ràng buộc vật lý được đặc tả tập trung bên trong một tập tin cấu hình định dạng YAML (.yaml). Tập tin này đóng vai trò như một bản thiết kế vi mạch kỹ thuật số, thiết lập các tham số cốt lõi để hls4ml phân tích đồ thị tính toán và đưa ra các quyết định kiến trúc tối ưu hóa mạch điện phù hợp với đặc tính tài nguyên hữu hạn trên chip.

Trong tập tin cấu hình YAML, một trong những thiết lập quan trọng nhất là việc định nghĩa chính xác hệ thống kiểu dữ liệu dấu phẩy thập phân cho từng thành phần bên trong mạng Autoencoder. Dựa trên các kết quả lượng tử hóa nhận thức từ QKeras, cấu hình toán học được ánh xạ đồng bộ sang cấu trúc định kiểu `ap_fixed` của Xilinx. Cụ thể, hệ thống tiến hành cấu hình ép kiểu cố định các biến trọng số, độ lệch và dữ liệu kích hoạt luân chuyển giữa các lớp về định dạng 8-bit, đảm bảo sự nhất quán về chiều rộng băng thông với các bộ nhân cứng vật lý. Ngoài ra, tập tin YAML cũng cho phép kỹ sư đặc tả chính xác tần số xung nhịp mục tiêu (Target Clock Frequency) cho bộ gia tốc, thông thường được thiết lập ở mức ổn định 100 MHz với chu kỳ xung nhịp định mức là 10 ns. Việc áp đặt ràng buộc thời gian khắt khe này buộc trình biên dịch Vitis HLS phải tính toán chu trình đi của dữ liệu một cách tối ưu, tự động chèn các thanh ghi dịch đệm hợp lý để giảm thiểu các nguy cơ vi phạm thời gian trễ đường truyền trên bề mặt silicon của chip Artix-7.

Sau khi hoàn tất việc khai báo các thông số kỹ thuật trong file YAML, quy trình dịch mã tự động được kích hoạt thông qua việc thực thi chuỗi lệnh API của hls4ml trên môi trường mã lệnh Python. Trình biên dịch sẽ tiến hành đọc tệp cấu hình, giải mã đồ thị của mạng Autoencoder và tự động sinh ra toàn bộ không gian thư mục dự án cho Vitis HLS. Hệ thống tự động khởi tạo mã nguồn C++ chất

lượng cao bao gồm các cấu trúc hàm nơ-ron kết nối đầy đủ, các tệp tiêu đề chứa mảng hằng số trọng số đầu phẩy tính và đặc biệt là mã lệnh dòng lệnh Tcl (Tool Command Language). Tập lệnh Tcl này sẽ tự động gọi luồng thực thi ẩn của Vitis HLS để chạy pha tổng hợp logic (C Synthesis) mà không cần kỹ sư phải thao tác thủ công trên giao diện đồ họa. Chu trình tự động hóa khép kín này không chỉ rút ngắn tối đa thời gian phát triển bộ gia tốc phần cứng mà còn bảo đảm sự đồng bộ toàn vẹn về mặt chức năng toán học từ mô hình mô phỏng xuống tầng cấu trúc mạch điện vật lý.

3.3.2 Giải quyết bài toán Routing Congestion bằng giao thức truyền luồng dữ liệu nối tiếp

Khi tiến hành hiện thực hóa kiến trúc mạng Autoencoder lên tầng vi mạch RTL, phương thức đóng gói và giao tiếp dữ liệu ngõ vào đóng vai trò quyết định đến độ khả thi của luồng định tuyến vật lý. Trong cấu hình mặc định ban đầu, nếu hệ thống sử dụng giao thức truyền dữ liệu mảng song song (io_parallel), toàn bộ véc-tơ ngõ vào phẳng gồm 196 phần tử sẽ được cấp phát đồng thời vào chip trong cùng một chu kỳ xung nhịp. Xét dưới góc độ tầng vật lý bán dẫn, việc truyền song song một ma trận dữ liệu đã lượng tử hóa 8-bit sẽ bắt buộc công cụ tổng hợp phải khởi tạo một tuyến bus phần cứng rất lớn có bề rộng lên tới 1568 đường dây dẫn tín hiệu cơ học chạy song song. Kiến trúc này đòi hỏi một số lượng cực kỳ lớn các thanh ghi chốt Flip-Flop (FF) hoạt động đồng thời để bắt kịp biên độ tín hiệu ngõ vào. Khi luồng dữ liệu này phân phối sâu vào các tầng ẩn của bộ Mã hóa, hệ thống sẽ đối mặt với nguy cơ tràn tài nguyên chốt FF nghiêm trọng, đồng thời kích hoạt hiện tượng bùng nổ các bộ dồn kênh kết nối mềm, vượt quá năng lực đáp ứng không gian lõi của dòng chip Artix-7. Hậu quả trực tiếp là hiện tượng nghẽn mạch (Routing Congestion). Lượng dây dẫn quá dày đặc không chỉ làm giảm mạnh tốc độ xung nhịp mà còn gây lỗi tràn bộ nhớ, khiến chip không thể chứa hết thiết kế (fitting failure).

Để khắc phục, thiết kế chuyển sang giao thức truyền dữ liệu nối tiếp (io_stream). Cơ chế này thiết lập một cấu hình phần cứng thông minh dựa trên việc kết hợp các hàng đợi FIFO nội bộ thông qua đối tượng khai báo hls::stream của thư viện Xilinx. Thay vì cưỡng bức hệ thống tiêu tốn tài nguyên để tiếp nhận toàn

bộ 196 điểm ảnh cùng một lúc, cấu trúc io_stream cho phép các giá trị cường độ điểm ảnh luân chuyển tuần tự, nối đuôi nhau di chuyển qua cổng nạp dữ liệu như một dòng chảy liên tục. Hệ thống phần cứng lúc này chỉ cần duy trì một đường bus dữ liệu hẹp có độ rộng đúng 8-bit dành cho một điểm ảnh, đi kèm các tín hiệu bắt tay kiểm soát luồng cơ bản bao gồm tín hiệu sẵn sàng (ready) và tín hiệu hợp lệ (valid). Sự tinh giản kiến trúc này đã giải phóng áp lực cho hệ thống dây dẫn định tuyến, khắc phục hiện tượng nghẽn mạch cục bộ, kìm hãm sự bùng nổ của các cổng logic mềm và hạ thấp đáng kể dung lượng tiêu thụ thanh ghi FF. Nhờ dòng dữ liệu gói đầu tuần tự qua mạch FIFO, bộ gia tốc mạng Autoencoder không chỉ hoạt động ổn định ở tần số mục tiêu cao mà còn đảm bảo luồng biên dịch RTL diễn ra trơn tru, vượt qua các ràng buộc vật lý nghiêm ngặt của linh kiện xc7a100tcs324-1.

3.3.3 Tối ưu hóa toán học bộ cộng dồn tại các nút nơ-ron

Một trong những thách thức kỹ thuật trọng yếu khi triển khai các mạng nơ-ron mật độ kết nối cao trên vi mạch là kiểm soát sự sai lệch toán học trong quá trình tích lũy đại số. Tại mỗi nút nơ-ron của các lớp QDense, mạch điện phải thực thi liên tục phép toán nhân chập tích lũy (MAC) giữa véc-tơ ngõ vào và ma trận trọng số, sau đó cộng dồn với giá trị độ lệch. Mặc dù các tham số trọng số và kích hoạt ngõ vào đã được tối ưu hóa nghiêm ngặt ở định dạng số dấu phẩy tĩnh 8-bit, việc thực thi phép cộng tổng của hàng trăm tích số thành phần (đặc biệt là tại tầng ngõ vào với 196 kết nối đồng quy) sẽ làm bùng nổ biên độ dữ liệu một cách tuyến tính. Nếu tiếp tục duy trì kiểu dữ liệu 8-bit cho bộ cộng dồn (accumulator), hệ thống sẽ ngay lập tức đối mặt với lỗi tràn số hệ thống (arithmetic overflow), làm sai lệch hàm ý toán học của thuật toán. Ngược lại, nếu sử dụng kiểu dữ liệu quá lớn như số dấu phẩy động 32-bit truyền thống, ưu thế về mặt diện tích và tốc độ của các lõi cứng DSP48E1 chuyên dụng trên chip Artix-7 sẽ không thể hiện được.

Để tháo gỡ điểm nghẽn này, nghiên cứu thực thi chiến lược tối ưu hóa vi kiến trúc bằng cách ép cấu trúc toán học của các bộ tích lũy trong mã nguồn C++ sang định dạng số dấu phẩy tĩnh cấu hình hệ thống ap_fixed<12,4>. Định dạng này thiết lập một thanh ghi có độ rộng vừa vặn là 12-bit, trong đó phân bổ chiến lược 4-bit dành cho phần nguyên (bao gồm cả bit dấu) và 8-bit dành cho phần phân số

thập phân. Việc mở rộng không gian phần nguyên lên 4-bit cung cấp một dải động đủ lớn để hấp thụ toàn bộ sự gia tăng biên độ khi hàng trăm luồng dữ liệu kết nối cùng hội tụ về một nơ-ron, ngăn ngừa các nguy cơ tràn bit ở các phép toán trung gian trước khi đưa vào hàm kích hoạt phi tuyến.

Đồng thời, việc cấu hình phần phân số chiếm tỉ trọng 8-bit đóng vai trò quyết định trong việc loại bỏ hiện tượng sụt giảm độ chính xác do cơ chế làm tròn về không (rounding towards zero hoặc underflow). Trong quá trình tính toán song song diện rộng, các giá trị trọng số cực nhỏ sau lượng tử hóa khi nhân với dữ liệu ngõ vào thường sinh ra các phần tử tích số có phần thập phân rất dài. Nếu bộ tích lũy không đủ độ phân giải, các giá trị này sẽ bị mạch logic hiểu sai là số không và cắt gọt thô bạo, dẫn đến hiện tượng sai số tích lũy tăng dần qua từng lớp ẩn và làm suy giảm nghiêm trọng năng lực phục dựng ảnh của mạng Autoencoder.

Định dạng `ap_fixed<12,4>` không chỉ bảo toàn tính toán vẹn toán học cho mô hình mà còn mang ý nghĩa tối ưu hóa phần cứng sâu sắc: cấu trúc 12-bit này ánh xạ vào bộ cộng dồn 48-bit nguyên bản bên trong lõi silicon DSP48E1. Trình tổng hợp Vitis HLS nhờ đó có thể triển khai mạch điện mà không cần chấp vá thêm bất kỳ cổng logic LUT mềm nào từ bên ngoài để mở rộng bit, giúp tối thiểu hóa diện tích lát cắt vật lý, giữ vững tần số hoạt động đỉnh cao 100 MHz và đảm bảo luồng suy luận đạt độ chuẩn xác tiệm cận mô hình phần mềm.

3.4 GIAO THỨC MÔ PHỎNG VÀ KIỂM CHỨNG

Để đảm bảo tính toàn vẹn và độ tin cậy của hệ thống trước khi nạp tệp bitstream thực tế lên lõi vi mạch, quy trình mô phỏng lập và kiểm chứng đồng bộ hóa phần cứng - phần mềm được thiết lập chặt chẽ thành ba giai đoạn liên hoàn bên trong hệ sinh thái Xilinx. Giai đoạn đầu tiên đóng vai trò nền tảng là thực thi mô phỏng mức phần mềm (C-Simulation) trên môi trường Vitis HLS. Mục tiêu cốt lõi của bước này là đánh giá và đối chiếu nhanh chất lượng toán học của mã nguồn C++ vừa được hls4ml sinh ra so với nguyên mẫu thuật toán (golden model) gốc trên Python. Việc kiểm chứng sớm này giúp kỹ sư xác nhận rằng cấu trúc mạng Autoencoder cùng các cấu hình ép kiểu dữ liệu dấu phẩy thập phân bảo toàn được năng

lực phục dựng hình ảnh, từ đó giảm thiểu rủi ro sai lệch khi chuyển sang giai đoạn hiện thực trên phần cứng.

Sau khi thuật toán phần mềm được xác thực độ chính xác, luồng thiết kế bước vào giai đoạn tổng hợp logic (C/RTL Synthesis). Tại pha này, công cụ Vitis HLS tiến hành phân tích phương pháp định tuyến, chuyển đổi toàn bộ mã nguồn C/C++ trừu tượng và ánh xạ trực tiếp thành các cấu trúc mạch điện mức chuyển thanh ghi (RTL) bao gồm các bộ dồn kênh, máy trạng thái và mạng lưới tính toán. Kết quả trả về của quá trình này là tập hợp các tập tin mô tả phần cứng Verilog/VHDL, đi kèm với một báo cáo ước lượng về diện tích tài nguyên tiêu thụ (LUT, FF, DSP, BRAM) cũng như dự toán tần số xung nhịp hoạt động trên không gian chip Artix-7 mục tiêu.

Giai đoạn cuối cùng để đánh giá hiệu năng thực tế của vi mạch là thực hiện kiểm chứng đồng bộ mô phỏng (C/RTL Co-Simulation). Thay vì chỉ ước lượng tĩnh, bước kiểm chứng này khai thác sức mạnh của trình mô phỏng phần cứng Vivado Simulator để kích thích trực tiếp lưới mạch RTL vừa tổng hợp bằng tập dữ liệu kiểm thử (testbench) C++. Cơ chế mô phỏng chu kỳ chính xác (cycle-accurate simulation) này tái hiện lại môi trường vật lý trên silicon, ép mạch điện phải tính toán và luân chuyển luồng dữ liệu tuần tự giống hệt như khi đang chạy trên phần cứng thật. Kết quả mô phỏng không chỉ xác minh lại tính đúng đắn của dữ liệu ngõ ra mà còn cho phép đo đạc chính xác số lượng chu kỳ xung nhịp thực tế (clock cycles/latency) cần thiết để bộ gia tốc xử lý hoàn chỉnh một ma trận bức ảnh. Các số liệu đo lường độ trễ và thông lượng từ pha Co-Simulation xác nhận kiến trúc thiết kế đã đáp ứng các ràng buộc về thời gian thực của hệ thống trí tuệ nhân tạo biên.

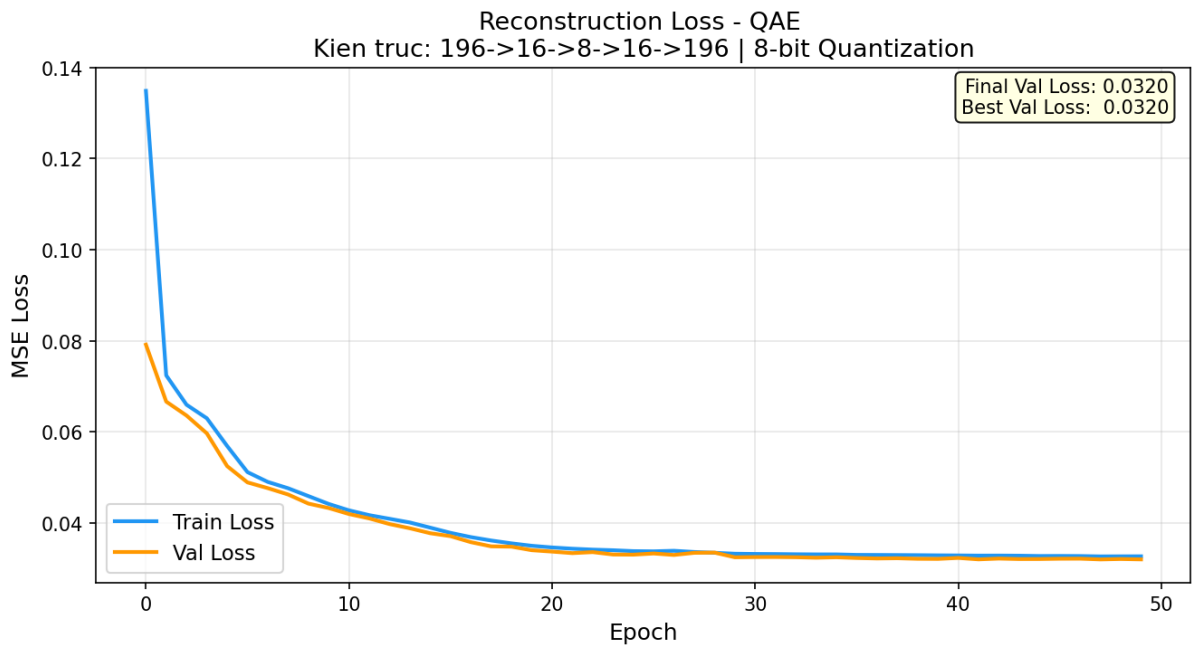
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU VÀ THẢO LUẬN

4.1 KẾT QUẢ HUẤN LUYỆN VÀ ĐÁNH GIÁ TRÊN PHẦN MỀM

Trước khi tiến hành chuyển đổi và triển khai tổng hợp trên vi mạch phần cứng FPGA, mô hình Autoencoder lượng tử hóa (QAE) được huấn luyện và đánh giá toàn diện trên môi trường phần mềm (CPU). Giai đoạn này đóng vai trò tiên quyết nhằm xác thực tính đúng đắn của thuật toán, kiểm tra khả năng hội tụ của mạng nơ-ron dưới các ràng buộc khắt khe về mặt độ rộng bit, đồng thời thiết lập hệ thống các chỉ số cơ sở (baseline metrics) để đối chiếu với phiên bản phần cứng sau này. Cấu trúc mạng được thử nghiệm sở hữu kiến trúc thu hẹp 196 $\rightarrow$ 16 $\rightarrow$ 8 $\rightarrow$ 16 $\rightarrow$ 196 kết hợp cùng kỹ thuật lượng tử hóa nhận thức 8-bit đồng bộ trên toàn bộ hệ thống trọng số và giá trị kích hoạt.

4.1.1 Đồ thị hội tụ (Loss) và các chỉ số định lượng

Phân tích chi tiết đồ thị đường cong học tập (learning curve) thu được từ quá trình huấn luyện mô hình mạng Tự mã hóa lượng tử hóa (QAE) cho thấy sự hội tụ ổn định của thuật toán. Trong giai đoạn 10 kỷ nguyên (epochs) đầu tiên, cả sai số trên tập huấn luyện (Train Loss) và tập xác thực (Val Loss) đều có xu hướng giảm mạnh, từ mức khởi điểm lần lượt là 0.135 và 0.08 lao xuống dưới ngưỡng 0.05. Sự sụt giảm đồng pha và nhanh chóng này minh chứng cho việc mô hình đang học hỏi đúng hướng, trích xuất thành công các đặc trưng hình thái lõi của tập ảnh chữ số MNIST bất chấp rào cản từ nút thắt cổ chai không gian tiềm ẩn. Bước sang pha bão hòa từ kỷ nguyên thứ 10 đến 50, đường cong Train Loss và Val Loss tiệm cận ngang và bám sát nhau, chốt lại với giá trị sai số tốt nhất (Best Val Loss) ở mức 0.0320. Khoảng cách hẹp giữa hai đường cong cho thấy mô hình không xảy ra hiện tượng quá khớp (overfitting) - điều này là tự nhiên với kiến trúc cực kỳ thu gọn (196 $\rightarrow$ 16 $\rightarrow$ 8 $\rightarrow$ 16 $\rightarrow$ 196), vì số lượng tham số quá ít để ghi nhớ dữ liệu huấn luyện.



Hình 4.1 Biểu đồ đường cong học tập thể hiện sự hội tụ của hàm mất mát (MSE Loss) qua 50 kỷ nguyên huấn luyện.

Bên cạnh biểu đồ hội tụ, chất lượng phục dựng ảnh của mô hình được đánh giá thông qua hai chỉ số định lượng. Chỉ số sai số toàn phương trung bình (MSE) trên tập kiểm tra (test set) đạt 0.030 và tỷ số tín hiệu trên nhiễu đỉnh (PSNR) đạt 15.18 dB. Giá trị PSNR này thấp hơn ngưỡng thông thường trong xử lý ảnh đa mục đích (thường yêu cầu >30 dB), tuy nhiên phù hợp với mục tiêu của hệ thống: đây không phải bài toán tái tạo ảnh chất lượng cao mà là nén và trích xuất đặc trưng tại biên với kiến trúc cực kỳ thu gọn (bottleneck 8 nơ-ron, trọng số 8-bit). Ở mức PSNR này, cấu trúc nét chính của chữ số vẫn được bảo toàn đủ để phân biệt nhân, như thể hiện qua Hình 4.2, tạo cơ sở để tiến hành tổng hợp vi mạch ở các giai đoạn tiếp theo.

4.1.2 Bản đồ sai số trực quan

Để đánh giá toàn diện chiều sâu và chất lượng giải mã của mạng Autoencoder lượng tử hóa (QAE), bên cạnh các chỉ số định lượng thuần túy (MSE, PSNR), phương pháp phân tích bản đồ sai số trực quan (Diff Map) đã được áp dụng nhằm mô tả đặc tính phân bố lỗi cục bộ của mô hình. Cấu trúc không gian

hình ảnh khảo sát trong Hình 4.2 được tổ chức thành ba dải thông tin đối chiếu trực tiếp. Hàng trên cùng hiển thị dữ liệu ảnh gốc (Ground Truth) trích xuất ngẫu nhiên từ tập kiểm thử. Hàng ở giữa thể hiện kết quả giải mã tái tạo từ ngõ ra của mô hình lượng tử hóa. Hàng dưới cùng là bản đồ nhiệt (heatmap) biểu diễn giá trị chênh lệch về cường độ điểm ảnh giữa nguyên bản và ảnh tái tạo. Đánh giá tổng quan trên hai hàng đầu tiên cho thấy, mặc dù luồng dữ liệu tín hiệu phải đi qua một không gian tiềm ẩn (latent space) cực kỳ chật hẹp (chỉ 8 nơ-ron) và chịu sự bóp méo nghiêm trọng của quá trình ép kiểu số học dấu phẩy tĩnh 8-bit, hệ thống vẫn giữ được các đặc trưng cấu trúc chính của chữ số. Hình dạng tổng thể và nét uốn đặc trưng phân biệt từng chữ số vẫn nhận dạng được, mặc dù có sai số tại các vùng rìa như sẽ phân tích ở bản đồ Diff Map.



Hình 4.2 Đánh giá trực quan kết quả suy luận của mô hình QAE trên tập dữ liệu MNIST (Từ trên xuống: Ảnh gốc đầu vào, Ảnh giải mã tái tạo, và Bản đồ nhiệt biểu diễn mức độ sai số).

Tiến hành phân tích sâu vào dải bản đồ nhiệt ở hàng cuối cùng, một đặc tính phân bố sai số mang tính hệ thống đã bộc lộ rất rõ nét. Trên bản đồ nhiệt, các điểm sai số lớn (màu đỏ, cam) không nằm rải rác mà tập trung dọc theo viền nét chữ. Thay vào đó, chúng quần tụ một cách có chủ đích, chạy dọc theo các đường biên và rìa viền chuyển tiếp của nét chữ. Ngược lại, tại khu vực lõi trung tâm của các nét vẽ, sai số có xu hướng giảm thiểu rõ rệt và phân bố đồng nhất hơn. Điều này xảy ra do mô hình đã học được cách ưu tiên bảo toàn các đặc trưng có cường độ tín hiệu cao và ổn định, vốn chiếm phần lớn diện tích không gian của nét chữ, từ

đó đảm bảo khả năng nhận diện hình thái ký tự ngay cả khi biên nét vẽ bị suy giảm độ phân giải sau quá trình lượng tử hóa.

Hiện tượng "mờ biên sinh lỗi" này đồng nhất với bản chất toán học của hệ thống phần cứng được thiết kế. Nguyên nhân thứ nhất bắt nguồn từ quá trình lấy mẫu xuống (downsampling) kết hợp với hiệu ứng làm tròn số (truncation) của thuật toán lượng tử hóa 8-bit, đã làm suy hao nghiêm trọng các thành phần dải tần số cao (high-frequency components) - vốn là những dải mang tín hiệu về độ sắc nét của các đường viền. Nguyên nhân thứ hai và cũng là nguyên nhân cốt lõi, nằm ở đặc tính nội tại của hàm mục tiêu sai số toàn phương trung bình (MSE) được sử dụng trong pha huấn luyện. Hàm MSE luôn có xu hướng phạt rất nặng các điểm dị biệt, dẫn đến việc mô hình sẽ tự động san phẳng các giá trị dự đoán tại vùng chuyển tiếp (từ nền đen sang nét trắng) để giảm thiểu chi phí phạt (penalty cost). Kết quả là mô hình chấp nhận làm mờ phần viền để giảm sai số ở vùng trung tâm nét chữ.

Tổng kết lại, sự phân bố sai số mang tính cục bộ ở rìa nét chữ được quan sát trên bản đồ Diff Map là một sự đánh đổi có thể dự đoán trước và chấp nhận được trong luồng thiết kế Edge AI. Dạng sai số này không làm phá vỡ cấu trúc tổng thể của dữ liệu gốc, qua đó tái khẳng định tính bền vững và sự tối ưu của thuật toán Lượng tử hóa nhận thức trên môi trường mô phỏng phần mềm. Đây chính là cơ sở an toàn và vững chắc để tiến hành quy trình tổng hợp mạch logic (C/RTL Synthesis) trên lõi FPGA ở các giai đoạn tiếp theo.

4.1.3 Khảo sát thời gian thực thi trên CPU laptop

Để thiết lập cơ sở dữ liệu tham chiếu (baseline) khách quan đánh giá hiệu năng bộ gia tốc phần cứng, thời gian suy luận (inference latency) của mô hình mạng Autoencoder lượng tử hóa được đo lường trực tiếp trên CPU. Nhằm loại bỏ các hao phí thời gian (overhead) phát sinh từ môi trường thông dịch Python và thư viện Keras/TensorFlow, mô hình được biên dịch sang định dạng ONNX (Open Neural Network Exchange) và thực thi qua bộ máy ONNX Runtime lõi C++.

Quy trình kiểm thử sử dụng hàm `time.perf_counter()`. Để loại bỏ nhiễu ngẫu nhiên từ quá trình chuyển đổi ngữ cảnh (context switching) của hệ thống, code

thực hiện chạy mỗi (warm-up) 10 vòng, sau đó đo tổng thời gian của 50 vòng lặp liên tục và lấy giá trị trung bình.

Kết quả đo lường ghi nhận, ở trường hợp đo thời gian thực cho một bức ảnh đơn lẻ (Batch = 1), độ trễ xử lý trên CPU đạt mức 0.02882 ms (tương đương 28.82 μ s). Mặc dù công cụ ONNX Runtime đã tinh giản cấu trúc phần mềm, thời gian thực thi này vẫn đại diện cho điểm nghẽn của kiến trúc Von Neumann. Để thực thi các phép toán nhân chập tích lũy (MAC), CPU buộc phải liên tục thực hiện chu kỳ nạp/lưu (load/store) ma trận trọng số từ bộ nhớ RAM thông qua các cấp bộ đệm. Sự giới hạn về khả năng xử lý song song không gian khiến CPU không thể phân giải toàn bộ 196 điểm ảnh đầu vào trong một chu kỳ xung nhịp. Mức độ trễ 28.82 μ s thiết lập cơ sở dữ liệu quan trọng, đặt ra yêu cầu phải sử dụng vi mạch gia tốc FPGA để giảm độ trễ xuống.

4.2 KẾT QUẢ TỔNG HỢP VÀ GIA TỐC TRÊN PHẦN CỨNG

4.2.1 Đánh giá mức độ tiêu thụ tài nguyên vật lý

Để đánh giá chính xác mức tiêu thụ tài nguyên và năng lượng của bộ gia tốc, toàn bộ mã RTL sinh ra từ Vitis HLS được đóng gói thành một lõi IP (Intellectual Property) độc lập và đưa sang Xilinx Vivado 2022.2. Tại đây, quá trình đánh giá được thực hiện bằng luồng Tổng hợp độc lập (Out-of-Context Synthesis). Cách làm này quan trọng về mặt phương pháp: thay vì phải dựng một hệ thống hoàn chỉnh kèm vi xử lý trung tâm, luồng Out-of-Context cho phép cô lập riêng lõi mạng nơ-ron để tổng hợp logic mức cổng, nhờ đó trích xuất được báo cáo chính xác về diện tích vật lý và điện năng tiêu thụ của riêng thuật toán. Kết quả thống kê tài nguyên được trình bày trong Bảng 4.1.

Bảng 4.1 Thống kê mức độ tiêu thụ tài nguyên trên vi mạch Artix-7 (xc7a100tcs g324-1).

Loại tài nguyên	Chức năng cốt lõi	Số lượng sử dụng	Tổng khả dụng	Tỷ lệ chiếm dụng (%)
DSP48E1	Thực thi phép toán nhân cộng MAC	120	240	50.00%
Block RAM (BRAM)	Lưu trữ trọng số và đệm dữ liệu	64	135	47.41%
LUT (Look-Up Table)	Cổng logic tổ hợp, máy trạng thái	15,931	63,400	25.13%
FF (Flip-Flop / Register)	Thanh ghi đồng bộ dòng thời gian	28,566	126,800	22.53%

Số liệu ở Bảng 4.1 cho thấy mức tiêu thụ tài nguyên phân bố không đều giữa bốn nhóm. Hai khối chuyên dụng là DSP (50.00%) và BRAM (47.41%) chịu tải cao nhất, trong khi tài nguyên logic nền tảng gồm LUT (25.13%) và FF (22.53%) chỉ ở mức khoảng một phần tư. Phân bố này phản ánh đúng đặc thù của thuật toán học sâu, vốn tập trung vào phép nhân ma trận và lưu trữ trọng số.

Về khối nhân cứng DSP, hệ thống sử dụng đúng một nửa số khối khả dụng (120 trên 240). Việc ép kiểu thuật toán xuống định dạng dấu phẩy tĩnh cho phép trình tổng hợp ánh xạ trực tiếp các phép nhân của mạng nơ-ron vào khối DSP48E1, thay vì ghép nối từ các cổng logic mềm. Tỷ lệ 50% đủ để thiết lập đường ống tính toán song song cho băng thông xử lý cao, đồng thời vẫn còn dư tài nguyên để tích hợp thêm vi xử lý mềm (như MicroBlaze) hoặc các ngoại vi giao tiếp trong một hệ thống SoC hoàn chỉnh.

Về bộ nhớ BRAM, hệ thống kích hoạt 64 khối loại 36 Kb (tương đương 128 khối 18 Kb theo cách đếm của Vivado), chiếm 47.41% trong tổng số 135 khối của chip Artix-7. Toàn bộ ma trận tham số gồm 6.764 trọng số cùng các véc-tơ kích

hoạt trung gian được nhúng tĩnh ngay cạnh các khối DSP. Nhờ cách phân bổ này, quá trình trao đổi dữ liệu với bộ nhớ ngoài (off-chip) được loại bỏ trong pha suy luận, giúp hệ thống giảm điểm thất cổ chai của kiến trúc von Neumann và duy trì xung nhịp hoạt động ổn định, không bị gián đoạn bởi độ trễ truy xuất.

Về tài nguyên logic nền tảng, LUT và FF đều duy trì ở mức thấp, khoảng 22–25%. Đây là kết quả của việc lượng tử hóa: do dữ liệu được giới hạn ở độ rộng bit nhỏ, kích thước của các bộ dồn kênh (Multiplexer) và mạch cộng dồn (Adder) tại các nút chuyển tiếp được hạn chế, nhờ đó luồng định tuyến (routing) của thiết kế gọn và mức tiêu thụ logic thấp.

Cuối cùng, cần làm rõ một hiện tượng trong báo cáo liên quan đến số chân I/O. Thông số sử dụng chân I/O vật lý được ghi nhận tới 6.282 chân, đạt tỷ lệ 2991.43% và vượt xa mức 210 chân khả dụng của dòng Artix-7. Đây không phải lỗi thiết kế logic mà là hiện tượng thường gặp trong pha Tổng hợp độc lập (Out-of-Context Synthesis). Nguyên nhân là thiết kế hiện đang trích xuất cổng giao tiếp dưới dạng mảng song song (Parallel Array) cho toàn bộ 196 điểm ảnh ngõ vào và các tín hiệu ngõ ra, thay vì đóng gói qua giao thức bus nối tiếp như AXI4-Stream hoặc AXI4-Lite [14]; do đó Vivado tạm thời ánh xạ mỗi bit dữ liệu thành một chân vật lý riêng. Ở bước tích hợp hệ thống (System Integration) tiếp theo, các tín hiệu này sẽ được bọc (wrapped) dưới dạng giao diện chuẩn, đưa tỷ lệ sử dụng I/O về giới hạn an toàn để nạp thành công bitstream xuống phần cứng.

thành công và tối ưu hóa vị trí đặt tập trung tại vùng 2/3 phía dưới của die chip Artix-7 (clock region X0Y0–X1Y2), trải đều trên cả hai cột clock region (X0 và X1), trong khi vùng clock region Y3 phía trên gần như không sử dụng. Việc tài nguyên được gom nhóm hợp lý không chỉ phản ánh chất lượng của luồng Place & Route, mà còn là bằng chứng trực quan cho thấy thiết kế đáp ứng chặt chẽ các ràng buộc định thời (timing constraints) tại tần số 100 MHz, đảm bảo tín hiệu truyền dẫn không bị vi phạm trễ pha. Kết quả này thiết lập nền tảng kỹ thuật vững chắc cho bước tạo tệp cấu hình bitstream, sẵn sàng nạp xuống phần cứng thực tế.

Quan sát chi tiết tại cửa sổ Device View trong Hình 4.3, ta thấy rõ các khối logic và thanh ghi được phân bố phân cấp theo kiến trúc luồng dữ liệu (dataflow). Trong Vivado 2022.2, toàn bộ tài nguyên đã được đặt (placed) — bao gồm LUT/FF (Slice), DSP48E1 và BRAM — đều được hiển thị bằng cùng một màu cyan (#37CACC) theo quy định color scheme mặc định của công cụ. Dải màu vàng theo chiều dọc quan sát được ở cạnh trái và phải của die chip biểu thị nền của các I/O Bank (High Range), không phải tài nguyên DSP48E1. Mật độ cyan dày đặc tại các vùng clock region X0Y0–X1Y2 phản ánh trực quan tổng lượng tài nguyên được khai thác: 50% DSP (120/240 khối), 47,41% BRAM (128/270 khối 18 Kb) và 22–25% LUT/FF, được công cụ Place & Route xếp liền kề nhau nhằm rút ngắn chiều dài đường dẫn dữ liệu (wire length) và tối thiểu hóa điện năng tiêu thụ.

Đặc biệt, mật độ tài nguyên này là minh chứng rõ rệt cho việc chuyển đổi thành công đồ thị tính toán từ phần mềm sang phần cứng:

Về phân bố: Các khối DSP được xếp chồng theo cấu trúc chuỗi, tương ứng với luồng lan truyền xuôi (forward propagation) của mạng Autoencoder. Sự đồng bộ này cho thấy mô hình không bị phân tán ngẫu nhiên mà được tối ưu theo luồng tín hiệu, giúp các phép toán nhân cộng tích lũy (MAC) thực hiện liên tục mà không phải chờ đợi truy xuất dữ liệu từ xa.

Về mật độ tài nguyên (resource density): Mức chiếm dụng 50% DSP và 47,41% BRAM (như Bảng 4.1) hiện lên trên Device View như các "vùng nóng" (hotspots) tập trung, khẳng định rằng thiết kế của chúng ta không gây lãng phí tài nguyên lên toàn bộ bề mặt chip, mà để dành khoảng trống cho các đường truyền dữ liệu không bị chồng lấp. Điều này giúp hệ thống tránh được hiện tượng "xung đột định tuyến" — một lỗi thường gặp khi triển khai các mạng nơ-ron phức tạp lên dòng chip FPGA cấu hình thấp như Artix-7.

Tính xác thực: Đây chính là bằng chứng xác thực nhất khẳng định các con số 50% DSP và 47,41% BRAM ghi nhận trong Bảng 4.1 không chỉ là những giá trị ước lượng tĩnh, mà là kết quả của sự sắp đặt tối ưu trên kiến trúc vật lý. Việc đạt được trạng thái này xác nhận rằng mô hình QAE Dense 196-

16-8-16-196 đã được tinh chỉnh để phù hợp với giới hạn vật lý của xc7a100tcs324-1.

4.2.2 Đánh giá thông số định thời và Tốc độ gia tốc

Báo cáo ước lượng hiệu năng (Performance Estimates) trích xuất từ Vitis HLS cung cấp các tham số động học của thiết kế, được phân tích trên hai khía cạnh: khả năng đáp ứng định thời và thông lượng xử lý. Các số liệu cụ thể được tổng hợp trong Bảng 4.2.

Bảng 4.2 Thông số ước tính hiệu năng (Performance Estimates) của mô hình mạng tự mã hóa sau khi trải qua luồng tổng hợp mức cao HLS.

Timing (đơn vị: ns)

Clock	Target	Estimated	Uncertainty	Slack
ap_clk	10.00	7.180	2.70	0.120

Latency & Interval

Latency (cycles)	Latency (absolute)	Interval (cycles)	Type
91	0.910 μ s	64	dataflow

Về định thời (Timing), hệ thống được đặt chu kỳ xung nhịp mục tiêu (Target) 10.00 ns, tương ứng tần số 100 MHz. Bảng 4.2 ghi nhận độ trễ ước lượng (Estimated) của đường dẫn găng dài nhất là 7.180 ns, kèm sai số định thời cho phép (Uncertainty) 2.70 ns. Theo nguyên tắc kiểm tra thời gian tĩnh, tổng thời gian thực tế bằng tổng của hai giá trị này: $7.180 + 2.70 = 9.880$ ns. Do giá trị này nhỏ hơn chu kỳ mục tiêu 10.00 ns, hệ thống đạt thời gian dự trữ dương (Slack) bằng 0.120 ns. Mức dự trữ dương này cho thấy các tín hiệu lan truyền qua khối nhân DSP và hệ thống thanh ghi vẫn ổn định, đáp ứng ràng buộc định thời tại tần số 100 MHz.

Về độ trễ suy luận (Latency), Bảng 4.2 cho thấy hệ thống cần tối đa 91 chu kỳ xung nhịp để giải mã và phục dựng hoàn chỉnh một ảnh. Thời gian suy luận trên FPGA bằng tích của số chu kỳ tối đa và chu kỳ xung nhịp: $91 \times 10 \text{ ns} = 910 \text{ ns}$, tương đương 0.91 μ s - khớp với giá trị Latency tuyệt đối 0.910 μ s trong Bảng 4.2.

Khi đối chiếu với độ trễ trên CPU (28.82 μ s, đo bằng ONNX Runtime ở Mục 4.1.3), hệ số gia tốc bằng tỷ lệ giữa hai độ trễ: $28.82 / 0.91 \approx 31.7$. Như vậy, bộ gia tốc nhanh hơn CPU khoảng 31.7 lần. Kết quả này cho thấy điểm nghẽn bộ nhớ của kiến trúc von Neumann đã được khắc phục nhờ việc nhúng ma trận trọng số nội bộ ngay sát các lõi tính toán song song.

Về thông lượng, thông số quan trọng là khoảng thời gian khởi tạo (Interval) cố định ở 64 chu kỳ, kết hợp với chế độ thực thi luồng dữ liệu (dataflow) như Bảng 4.2 thể hiện ở cột Type. Nhờ kiến trúc đường ống (pipeline) cấp tác vụ, hệ thống không phải chờ đủ 91 chu kỳ xử lý xong ảnh hiện tại rồi mới nhận ảnh mới; sau mỗi 64 chu kỳ, khối ngõ vào đã sẵn sàng tiếp nhận ma trận dữ liệu tiếp theo. Thông lượng lớn nhất bằng nghịch đảo của khoảng khởi tạo: $1 / (64 \times 10 \text{ ns}) = 1.562.500$ ảnh/giây (hơn 1.56 triệu ảnh mỗi giây). Mức thông lượng này đáp ứng yêu cầu thời gian thực của các hệ thống Trí tuệ Nhân tạo tại biên và phù hợp với các ứng dụng TinyML.

4.2.3 Đánh giá Công suất tiêu thụ

Bên cạnh các yếu tố về không gian diện tích và tốc độ xử lý, công suất tiêu thụ điện năng (Power Consumption) là một chỉ tiêu mang tính quyết định để đánh giá mức độ khả thi của bộ gia tốc khi triển khai trên các thiết bị biên (Edge Devices) có giới hạn khắt khe về năng lượng. Quá trình phân tích năng lượng được thực thi thông qua công cụ phân tích công suất tích hợp của Vivado, dựa trên các tham số môi trường tiêu chuẩn với nhiệt độ môi trường (Ambient Temperature) ở mức 25°C.

Bảng 4.3 Phân bố công suất tiêu thụ động

Thành phần phần cứng	Công suất (W)	Tỷ trọng trong vùng Động (%)	Chức năng chính
DSPs (Bộ xử lý tín hiệu số)	0.129	22.7%	Thực thi phép toán nhân chập (MAC)

I/O (Cổng giao tiếp)	0.126	22.2%	Xuất/nhập dữ liệu song song (OOC)
Signals (Tín hiệu định tuyến)	0.125	22.0%	Luân chuyển dữ liệu nội bộ
Slice Logic (Logic mềm)	0.111	19.5%	Cổng LUT và thanh ghi
Clocks (Xung nhịp)	0.041	7.2%	Cây phân phối xung nhịp toàn cục
Block RAM (Bộ nhớ cục bộ)	0.037	6.5%	Lưu trữ trọng số và tín hiệu trung gian

Kết quả tổng hợp toàn cục cho thấy tổng công suất tiêu thụ trên toàn chip (Total On-Chip Power) chỉ ở mức 0.665 W. Mức năng lượng này được phân rã thành hai cấu phần cơ bản: công suất tĩnh (Device Static) chiếm 0.097 W do dòng rò bán dẫn nội tại của phôi chip Artix-7, và công suất động (Dynamic) chiếm 0.568 W phát sinh trực tiếp từ hoạt động chuyển mạch của thuật toán. Sự tối ưu về năng lượng này được phản ánh rõ nét nhất qua chỉ số nhiệt độ phân cực (Junction Temperature). Báo cáo ghi nhận nhiệt độ lõi silic chỉ nhích nhẹ lên mức 28.0°C so với 25°C của môi trường. Mức tăng chỉ 3°C cho thấy thiết kế tỏa nhiệt rất ít và ổn định về nhiệt, không cần tản nhiệt chủ động.

Đi sâu vào cấu trúc tiêu hao năng lượng động (Dynamic Power), các lõi nhân cộng DSP và mạng lưới tín hiệu định tuyến (Signals) là hai thành phần tiêu thụ điện chính, lần lượt 0,129 W và 0,125 W. Sự phân bổ này phù hợp với bản chất kiến trúc Dataflow đã được cấu hình. Việc hệ thống liên tục bơm dữ liệu qua hàng loạt đường ống song song để thực thi hàng ngàn phép tính MAC trong mỗi chu kỳ xung nhịp tất yếu sẽ tạo ra mức tiêu thụ năng lượng lớn tại các nút xử lý toán học và các đường bus kết nối nội bộ.

Trái ngược với mức tiêu thụ cao của lõi tính toán, khối bộ nhớ BRAM lại thể hiện đặc tính tiết kiệm năng lượng vượt trội khi chỉ tiêu tốn 0.037 W. Sự chênh lệch này cho thấy hiệu quả của phương pháp thiết kế nhúng trọng số nội bộ. Bằng việc đưa toàn bộ ma trận tham số vào khối BRAM thay vì truy xuất liên tục từ bộ nhớ DDR RAM bên ngoài, hệ thống đã cắt giảm được một lượng lớn điện năng hao phí do quá trình nạp/lưu tín hiệu cao tần qua lại trên bo mạch, qua đó trực tiếp kéo giảm tổng công suất toàn hệ thống.

Một điểm kỹ thuật cần được giải thích rõ trong báo cáo là mức tiêu thụ năng lượng cao bất thường của các khối I/O (đạt 0.126 W). Tương tự như hiện tượng quá tải chân cắm vật lý đã được phân tích ở mục 4.2.1, mức tiêu thụ điện năng này là hệ quả tất yếu của quá trình Tổng hợp độc lập (Out-of-Context Synthesis). Công cụ Vivado buộc phải kích hoạt hàng ngàn bộ đệm I/O ảo để đáp ứng việc đo lường mảng dữ liệu song song rất lớn. Tuy nhiên, thông số tiêu thụ 0.126 W này chỉ mang tính chất mô phỏng lý thuyết trong môi trường cô lập. Ở các giai đoạn tích hợp hệ thống cuối cùng, khi toàn bộ 196 điểm ảnh ngõ vào được đóng gói thành một luồng dữ liệu nối tiếp tuân thủ chuẩn AXI4-Stream, mức tiêu thụ năng lượng của khối I/O sẽ sụt giảm mạnh, đưa tổng công suất toàn chip về mức lý tưởng hơn nữa.

Tổng quan lại, mức tiêu hao 0.665 W cho một kiến trúc mạng nơ-ron đạt thông lượng 1.56 triệu khung hình/giây. Tỷ số hiệu năng/năng lượng này đáp ứng đúng mục tiêu tối ưu hóa mà đề án đặt ra ban đầu. Kết quả này chứng minh luận điểm: bộ gia tốc phần cứng dựa trên kỹ thuật lượng tử hóa 8-bit không chỉ bứt phá về tốc độ mà còn đáp ứng tiêu chí xanh, tiết kiệm năng lượng của kỷ nguyên Edge AI.

4.2.4 Tính toán Thông lượng Toán học

Bên cạnh các thông số về thời gian suy luận và công suất tiêu thụ điện năng, thông lượng toán học (Mathematical Throughput) là thước đo chuẩn mực và toàn diện nhất để đánh giá sức mạnh xử lý thô của một bộ gia tốc phần cứng mạng nơ-ron. Thông lượng này thường được định lượng bằng chỉ số GOP/s (Giga Operations Per Second - Tỷ phép toán trên giây). Để xác định năng lực thực thi của lõi IP vừa thiết kế, thông lượng toán học được định lượng bằng chỉ số tỷ phép toán trên giây (GOP/s). Chỉ số này được xác định bằng cách lấy tổng số lượng

phép toán đơn lẻ (bao gồm phép nhân và phép cộng) trong một chu kỳ suy luận nhân với tốc độ giải mã khung hình (FPS), sau đó quy đổi sang hệ tỷ phép toán. Dựa trên cấu trúc mạng Autoencoder đã thiết kế, mỗi chu kỳ truyền thẳng (forward pass) yêu cầu hệ thống thực thi 6.528 phép toán nhân cộng tích lũy (MAC), tương đương với 13.056 phép toán đơn lẻ. Kết hợp với thông lượng dữ liệu thực tế đã được khảo sát ở Mục 4.2.2 là 1.562.500 ảnh/giây, bộ gia tốc phần cứng đạt được thông lượng toán học thực tế ở mức 20,40 GOP/s. Kết quả này khẳng định hệ thống có khả năng xử lý lên tới 20,40 tỷ phép toán trong một giây. Đối với một hệ thống được triển khai trên dòng linh kiện FPGA tầm trung như Artix-7, đây là một mật độ tính toán cực kỳ ấn tượng. Con số này là minh chứng rõ nét cho thấy sự thành công của quy trình Đồng thiết kế Phần cứng - Phần mềm khi đã loại bỏ được những điểm nghẽn về băng thông dữ liệu, đồng thời khai thác triệt để tính song song không gian của các khối DSP chuyên dụng để đáp ứng các tiêu chuẩn khắt khe về xử lý thời gian thực.

Hơn thế nữa, giá trị thông lượng 20.40 GOP/s này còn mang ý nghĩa đặc biệt quan trọng khi được đặt trong hệ quy chiếu với năng lượng tiêu thụ. Bằng cách kết hợp thông lượng toán học với mức công suất tổng toàn chip (đạt 0,665 W như đã đo lường ở Mục 4.2.3), hệ thống cho phép đánh giá hiệu suất năng lượng bằng cách lấy tỷ phép toán trên giây chia cho tổng công suất tiêu thụ. Kết quả tính toán thực tế cho thấy tỷ số này đạt mức 30,68 GOP/s/W. Chỉ số này khẳng định rằng với mỗi một Watt điện năng tiêu thụ, bộ gia tốc có khả năng thực thi hơn 30 tỷ phép toán học. Tỷ số hiệu năng trên năng lượng này chính là lời giải đáp cho bài toán tối ưu hóa mà đề án đặt ra từ ban đầu.

4.3 ĐỐI CHIẾU HIỆU NĂNG GIỮA FPGA, CPU VÀ GPU

Sau khi đánh giá độc lập từng nền tảng, phần này đo và đối chiếu trực tiếp hiệu năng của ba kiến trúc triển khai: CPU Intel Core i5 chạy qua ONNX Runtime, GPU NVIDIA RTX 3050 Ti chạy qua TensorRT Python API, và lõi gia tốc FPGA Artix-7 100T đã phân tích ở Mục 4.2. Trình tự trình bày đi từ phương pháp đo, đến kết quả thực nghiệm, rồi mới lý giải các hiện tượng thu được dựa trên chính số liệu đã đo - qua đó làm rõ vì sao FPGA là lựa chọn phù hợp nhất cho bài toán suy luận thời gian thực trên thiết bị biên.

4.3.1 Phương pháp đo và cấu hình thực nghiệm

Để các kết quả so sánh được với nhau, toàn bộ phép đo cho CPU và GPU được gộp trong một tập lệnh Python duy nhất (`bench_cpu_gpu_trt.py`), chạy trên cùng một máy với cùng dữ liệu đầu vào, loại bỏ sai số do khác môi trường.

Với CPU, ONNX Runtime 1.23.2 chạy bằng `CPUExecutionProvider`, bật toàn bộ tối ưu đồ thị và thư viện nhân MLAS. Mỗi phiên đo có 100 vòng khởi động (warm-up) để mô hình nạp xong vào cache và hệ điều hành ổn định tài nguyên, sau đó đo 500 vòng liên tiếp bằng `time.perf_counter()` (độ phân giải nano-giây) rồi lấy trung bình. Việc dùng 500 vòng giúp triệt tiêu nhiễu ngẫu nhiên từ hệ điều hành như context switching và garbage collection.

Với GPU, cần làm rõ một vấn đề kỹ thuật. ONNX Runtime 1.23.2 có sẵn `TensorrtExecutionProvider`, nhưng trong môi trường thực nghiệm nó không dùng được do xung đột giao diện nhị phân (ABI): ORT 1.23.2 liên kết với TensorRT 10, trong khi máy đã cài TensorRT 11.1.0.106. Khi nạp, hệ thống lỗi thư viện động rồi âm thầm chuyển về CPU mà không báo - tức nhãn ghi "TRT" nhưng thực chất đang đo CPU, cho số liệu sai. Do đó đề tài gọi TensorRT Python API trực tiếp qua thư viện `tensorrt` và `pycuda`, đảm bảo engine TRT thực sự chạy trên GPU.

Engine TensorRT được dựng từ mô hình ONNX động (`qae196_dynamic.onnx`) với optimization profile phủ batch từ 1 đến 16.384. Để loại chi phí cấp phát bộ nhớ khởi phép đo, toàn bộ vùng đệm GPU được cấp phát một lần và tái sử dụng. Số vòng warm-up và đo giữ nguyên 100/500 như CPU.

Với FPGA, số liệu lấy trực tiếp từ báo cáo Performance Estimates của Vitis HLS 2022.2 (Mục 4.2.2): độ trễ $91 \text{ chu kỳ} \times 10 \text{ ns} = 0,91 \mu\text{s}/\text{ảnh}$, và thông lượng đường ống theo chu kỳ khởi tạo 64 chu kỳ là $1.562.500 \text{ ảnh}/\text{giây}$, cố định bất kể kích thước lô.

4.3.2 Kết quả đo tổng hợp

Bảng 4.4 Độ trễ theo kích thước lô - nhỏ hơn = tốt hơn ($\mu\text{s}/\text{ảnh}$)

Batch	CPU ORT	GPU TensorRT	FPGA Artix-7
1	28,82	41,27	0,91
32	1,34	1,54	0,91
128	0,57	0,56	0,91
512	0,29	0,34	0,91
1024	0,24	0,30	0,91

Bảng 4.5 Thông lượng theo kích thước lô - lớn hơn = tốt hơn (ảnh/giây)

Batch	CPU ORT	GPU TensorRT	FPGA Artix-7
1	34.700	24.228	1.562.500
32	746.300	649.400	1.562.500
128	1.763.700	1.785.700	1.562.500
512	3.424.700	2.941.200	1.562.500
1024	4.166.700	3.333.300	1.562.500

Kết quả ở hai bảng trên thể hiện ba đặc điểm chính. Ở chế độ đơn ảnh (Batch = 1), GPU TensorRT có độ trễ 41,27 μs , cao hơn CPU (28,82 μs), trái với kỳ vọng thông thường rằng GPU luôn nhanh hơn. Khi kích thước lô tăng, CPU và GPU luân phiên dẫn đầu: CPU đạt độ trễ thấp hơn ở Batch = 1 và 32, GPU thấp hơn ở Batch = 128, CPU tiếp tục thấp hơn ở Batch = 512 và 1024, với mức chênh lệch dưới 20% trên toàn dải. FPGA duy trì độ trễ 0,91 μs và thông lượng 1.562.500 ảnh/giây cố định ở mọi kích thước lô, thấp hơn đáng kể so với CPU và GPU ở chế độ đơn ảnh.

4.3.3 Phân tích nguyên nhân chênh lệch hiệu năng giữa CPU và GPU

Thông thường là GPU luôn nhanh hơn CPU nhờ hàng nghìn nhân tính toán song song. Điều này đúng khi mô hình đủ lớn, nhưng kết quả ở Bảng 4.4 và Bảng 4.5 cho thấy nó không đúng với QAE 196. Nguyên nhân nằm ở hai yếu tố: overhead cố định của GPU và mật độ tính toán của mô hình.

Overhead cố định của GPU. GPU không thể tính ngay khi nhận lệnh. Mỗi lần suy luận, hệ thống buộc phải chạy tuần tự một chuỗi thao tác chuẩn bị, bất kể mô hình lớn hay nhỏ: truyền dữ liệu từ RAM của máy chủ sang VRAM của GPU qua PCI Express (H2D), gửi lệnh khởi động nhân CUDA qua tầng driver, truyền kết quả ngược lại (D2H), và đồng bộ hóa (`stream.synchronize()`). Ngay cả khi một ảnh đầu vào chỉ nặng $196 \times 4 = 784$ byte, riêng việc thiết lập giao dịch trên bus PCIe đã tốn 1–3 μ s cố định, còn chuỗi giao tiếp phần mềm qua nhiều lớp của hệ điều hành và driver tốn thêm vài chục micro-giây. Chính tổng các chi phí này tạo nên con số 41,27 μ s đo được ở Batch = 1 trong Bảng 4.4 - toàn bộ là overhead, gần như không có phần tính toán hữu ích.

Mật độ tính toán của mô hình. Đây là tỉ số giữa số phép tính phải thực hiện và lượng dữ liệu phải đọc/ghi. Mật độ cao thì phần cứng tính được nhiều trong khi chỉ nạp ít dữ liệu - điều kiện lý tưởng để GPU phát huy song song; mật độ thấp thì phần cứng chủ yếu chờ dữ liệu thay vì tính. QAE 196-16-8-16-196 (đã tính ở Mục 3.2.4) chỉ cần 6.528 phép MAC, tương đương 13.056 phép tính dấu phẩy động cho một lần suy luận - một con số rất nhỏ. Để so sánh, ResNet-50 cần hơn 4 tỉ phép tính cho một ảnh 224×224 , tức gấp hơn 300.000 lần QAE 196.

Đặt hai yếu tố cạnh nhau sẽ thấy rõ nghịch lý. Với sức mạnh đỉnh 7,0 TFLOPS của RTX 3050 Ti, toàn bộ 13.056 phép tính của QAE 196 chỉ mất khoảng 0,0019 μ s ($\approx 1,9$ nano-giây) nếu bỏ qua overhead - nhỏ hơn 41,27 μ s overhead tới hơn 20.000 lần. Nói cách khác, GPU phải bỏ ra hơn 41 μ s để chuẩn bị cho chưa đến 0,002 μ s tính toán thật. Đó chính là gốc rễ khiến GPU chậm hơn CPU ở Batch = 1.

Ngược lại, CPU Intel Core i5 không cần thao tác chuẩn bị nào. Thư viện MLAS trong ONNX Runtime chạy toàn bộ phép nhân ma trận của QAE 196 ngay

trong bộ đệm L2 mà không qua bus ngoài. Toàn bộ 6.764 tham số chỉ chiếm khoảng 26 KB, nằm gọn trong L2 256 KB của Core i5, nên CPU tái sử dụng trọng số liên tục mà không phải nạp lại từ RAM. Vì vậy, cả 28,82 μ s mà CPU tiêu tốn ở Batch = 1 đều là thời gian tính toán có ích - giải thích vì sao CPU về đích trước GPU ở chế độ đơn ảnh.

4.3.4 Phân tích theo từng vùng kích thước lô

Đơn ảnh (Batch = 1). Đây là chế độ quan trọng nhất cho phát hiện bất thường thời gian thực trên thiết bị biên, nơi từng ảnh cần có kết quả ngay mà không chờ gom lô. Như đã lý giải ở Mục 4.3.3, 41,27 μ s của GPU hoàn toàn là overhead, còn 28,82 μ s của CPU là thời gian tính toán thật - nên CPU nhanh hơn GPU 1,43 lần.

Batch vừa (32–128). Khi lô lớn dần, overhead cố định của GPU được chia đều cho nhiều ảnh nên chi phí trung bình mỗi ảnh giảm. Tại Batch = 32, GPU đạt 1,54 μ s/ảnh so với CPU 1,34 μ s/ảnh - khoảng cách thu hẹp nhưng CPU vẫn thắng. Tại Batch = 128, GPU lần đầu đạt 0,56 μ s/ảnh so với CPU 0,57 μ s/ảnh - gần như ngang nhau, GPU chỉ nhỉnh 1,8%. Đây là điểm giao nhau đầu tiên giữa hai đường hiệu năng.

Batch lớn (512–1024). Bất ngờ là CPU lại vượt lên. Tại Batch = 512, CPU đạt 0,29 μ s/ảnh so với GPU 0,34 μ s/ảnh (CPU nhanh hơn 1,17 lần); tại Batch = 1024, CPU đạt 0,24 μ s/ảnh so với GPU 0,30 μ s/ảnh. Nguyên nhân nằm ở quản lý bộ nhớ: khi Batch = 512, dữ liệu đầu vào khoảng 400 KB, vượt L2 256 KB của CPU. Nhưng MLAS xử lý bằng chia khối (tiling) - cắt phép nhân ma trận lớn thành các khối vừa cache rồi tính tuần tự - nên vẫn tái sử dụng dữ liệu hiệu quả. GPU thì phải nạp cả lô vào VRAM qua PCIe trước khi tính, tạo điểm nghẽn băng thông đúng lúc CPU đang nhờ tiling mà cạnh tranh tốt hơn.

Tổng kết dải Batch 1–1024. Không có một ngưỡng lô cố định nào mà GPU TensorRT vượt CPU một cách bền vững với QAE 196; hai nền tảng thay nhau dẫn đầu với biên độ dưới 20% - không có ý nghĩa thực tiễn rõ ràng. Điều này khẳng định GPU TensorRT không có lợi thế thực chất so với CPU cho lớp mô hình Dense

siêu nhỏ; TensorRT chỉ thực sự phát huy trên các mạng tích chập mật độ cao như ResNet hay EfficientNet.

4.3.5 Đánh giá ưu thế của FPGA so với CPU và GPU

So với mức chênh lệch nhỏ giữa CPU và GPU TensorRT, FPGA Artix-7 cho hiệu năng cao hơn hẳn. Tại Batch = 1 (chế độ suy luận đơn ảnh thời gian thực), FPGA hoàn tất cả mã hóa và giải mã trong 0,91 μ s, nhanh hơn CPU 31,7 lần (28,82 / 0,91) và nhanh hơn GPU TensorRT 45,4 lần (41,27 / 0,91). Về thông lượng đường ống, FPGA đạt 1.562.500 ảnh/giây cố định, cao hơn CPU 45 lần và cao hơn GPU 64,5 lần ở cùng chế độ.

Tại Batch = 1024, CPU đạt 4.166.700 ảnh/giây và GPU đạt 3.333.300 ảnh/giây, cao hơn thông lượng của FPGA về tổng số ảnh xử lý mỗi giây. Tuy nhiên, cần phân biệt hai loại thông lượng. Thông lượng lô yêu cầu gom đủ 1.024 ảnh trước khi xử lý, làm độ trễ phản hồi thực tế tăng lên 0,24 ms với CPU và 0,31 ms với GPU. Ngược lại, FPGA giữ độ trễ 0,91 μ s cố định cho từng ảnh ngay khi ảnh đến, không cần chờ gom lô. Với các hệ thống kiểm soát công nghiệp và giám sát an toàn yêu cầu phản hồi tức thì cho từng sự kiện, độ trễ tất định 0,91 μ s của FPGA là yêu cầu mà thông lượng lô cao không thể thay thế.

Về năng lượng, chênh lệch giữa các nền tảng cũng rất lớn. GPU RTX 3050 Ti có công suất thiết kế nhiệt (TDP) 75 W, gấp 112 lần toàn hệ thống FPGA (0,665 W). Với thiết bị cảm biến biên chạy bằng pin hoặc thu hoạch năng lượng từ môi trường, mức tiêu thụ 75 W của GPU là không khả thi.

4.4 TIỀM NĂNG TRIỂN KHAI THỰC TẾ

4.4.1 Định hướng ứng dụng thực tiễn

Nhờ độ trễ thấp, tất định và mức tiêu thụ năng lượng nhỏ, thiết kế phù hợp với một số nhóm ứng dụng tiêu biểu trong lĩnh vực Trí tuệ Nhân tạo tại biên.

Trong giám sát công nghiệp thời gian thực, hệ thống thị giác máy trên các dây chuyền tốc độ cao cần phát hiện sản phẩm lỗi trên từng khung hình, không cho phép bỏ sót hoặc xử lý trễ. Độ trễ tắt định 0,91 μ s cho mỗi ảnh cho phép xử lý từng sản phẩm khi đi qua camera mà không cần gom lô, đáp ứng yêu cầu thời gian thực cứng (hard real-time) mà CPU và GPU khó bảo đảm do độ trễ thay đổi theo tải hệ thống.

Trong các ứng dụng nén dữ liệu tại nguồn trên cảm biến IoT, chẳng hạn thiết bị y tế đeo trên người hoặc cảm biến môi trường đặt tại hiện trường, mạng QAE nén tín hiệu hoặc ảnh trước khi truyền về trung tâm, qua đó giảm lưu lượng truyền qua mạng, tiết kiệm băng thông và kéo dài thời gian dùng pin. Việc xử lý tại chỗ cũng nâng cao tính bảo mật do dữ liệu thô không cần truyền ra ngoài thiết bị.

Đối với thiết bị biên ở khu vực thiếu nguồn điện ổn định, mức tiêu thụ 0,665 W cho phép hệ thống vận hành bằng pin hoặc nguồn thu hoạch năng lượng như pin mặt trời cỡ nhỏ, phù hợp cho camera an ninh, trạm quan trắc từ xa và thiết bị giám sát đặt ngoài lưới điện.

4.4.2 Lợi ích kỹ thuật

So với nền tảng phần mềm đa năng, thiết kế FPGA mang lại ba lợi ích kỹ thuật, đều rút ra từ số liệu đo được. Về thời gian đáp ứng, độ trễ 0,91 μ s có tính tắt định, đáp ứng các hệ thống thời gian thực cứng, trong khi CPU (28,82 μ s, dao động theo tải hệ điều hành) và GPU (41,27 μ s, chịu chi phí overhead) không bảo đảm được khung thời gian cố định. Về năng lượng, mức công suất 0,665 W thấp hơn GPU RTX 3050 Ti (75 W) khoảng 112 lần, cho phép triển khai dưới ràng buộc năng lượng chặt. Về tính độc lập, toàn bộ quá trình suy luận diễn ra tại chỗ nên hệ thống không phụ thuộc kết nối đám mây, loại bỏ độ trễ mạng và giảm rủi ro lộ dữ liệu trên đường truyền.

4.4.3 Chi phí, vận hành và vòng đời

Về chi phí điện năng, ở mức 0,665 W, thiết bị tiêu thụ khoảng 5,83 kWh nếu vận hành liên tục cả năm ($0,665 \text{ W} \times 24 \times 365 \approx 5,83 \text{ kWh}$), tương ứng chi phí điện không đáng kể. Cùng điều kiện, GPU RTX 3050 Ti ở mức 75 W tiêu thụ khoảng 657 kWh mỗi năm, cao hơn khoảng 110 lần. Với một mạng gồm nhiều node cảm biến hoạt động liên tục, chênh lệch này dẫn đến khác biệt rõ rệt về chi phí vận hành.

Về chi phí phần cứng, chip XC7A100T thuộc dòng Artix-7 tầm trung, được định hướng tối ưu chi phí. Tại thời điểm năm 2025, giá tham khảo khoảng 100–160 USD cho chip rời ở số lượng lẻ và khoảng 150 USD cho board phát triển dùng trong nghiên cứu, và giảm đáng kể khi mua số lượng lớn; mức giá thay đổi theo nhà phân phối và sản lượng. So với GPU rời cùng phân khúc, đây là mức chi phí phù hợp để nhân rộng trên nhiều thiết bị biên.

Về tản nhiệt và vòng đời, kết quả ở Mục 4.2.3 cho thấy nhiệt độ chip chỉ tăng khoảng 3°C so với môi trường, nên hệ thống không cần giải pháp tản nhiệt chủ động như quạt hay khối tản nhiệt lớn. Việc lược bỏ các linh kiện cơ khí giúp giảm chi phí và kích thước, đồng thời nâng cao độ tin cậy và tuổi thọ, phù hợp với môi trường công nghiệp có bụi, rung động hoặc nhiệt độ cao.

4.4.4 Hạn chế khi triển khai

Bên cạnh các ưu điểm trên, giải pháp FPGA cũng có một số hạn chế. Quy trình thiết kế đòi hỏi kiến thức về tổng hợp mức cao (HLS) và mô tả phần cứng mức thanh ghi (RTL), nên thời gian phát triển dài hơn so với lập trình trên CPU hoặc GPU. Mỗi lần thay đổi kiến trúc mô hình đều phải tổng hợp lại bitstream, kém linh hoạt hơn việc nạp lại một mô hình phần mềm. Ngoài ra, giải pháp phù hợp với các mô hình nhỏ, cố định và đã lượng tử hóa; nó không thay thế GPU trong huấn luyện hoặc trong các mô hình lớn có mật độ tính toán cao. Do đó, FPGA phù hợp với vai trò chuyên dụng cho suy luận tại biên, đóng vai trò bổ sung cho các nền tảng đa năng thay vì thay thế hoàn toàn.

4.5 PHÂN TÍCH KIẾN TRÚC QCNN MỞ RỘNG VÀ TÌNH TRẠNG TRÀN TỔNG HỢP FPGA

Trong khuôn khổ nghiên cứu mở rộng, nhóm tác giả đã thử nghiệm triển khai một kiến trúc mạng Tự mã hóa tích chập lượng tử hóa (QCNN ConvAE) với kích thước đầu vào ảnh đầy đủ 28×28 pixel nhằm đánh giá khả năng mở rộng của phương pháp và làm nền tảng so sánh với mô hình QAE Dense gốc đã được triển khai thành công trên FPGA Artix-7 100T.

4.5.1 Kiến trúc QCNN ConvAE Mở rộng

Kiến trúc mạng QCNN mở rộng được thiết kế theo cấu trúc Encoder – Latent – Decoder hoàn chỉnh. Bộ mã hóa gồm hai lớp QConv2D (16 và 8 bộ lọc 3×3) kết hợp MaxPooling2D ($28 \rightarrow 14 \rightarrow 7$ pixel), tiếp nối bởi lớp Flatten và QDense ($392 \rightarrow 16$) tạo không gian ẩn 16 chiều. Bộ giải mã thực hiện tái tạo qua QDense ($16 \rightarrow 392$), Reshape ($7 \times 7 \times 8$), hai lớp UpSampling2D ($7 \rightarrow 14 \rightarrow 28$), các lớp QConv2D (8 và 16 bộ lọc) và QConv2D đầu ra (1 bộ lọc, sigmoid). Toàn bộ lớp áp dụng lượng tử hóa 8-bit QKeras (quantized_bits(8,3,1), quantized_relu(8,3)), nhất quán với QAE Dense. Tổng số tham số: 16.169; số phép MAC: 1.480.192 (lớn hơn QAE Dense ~227 lần).

4.5.2 Kết quả Phần mềm - Mô hình chạy được trên CPU

Mô hình QCNN ConvAE được huấn luyện thành công bằng file train_qcnn_convae.ipynb trên tập dữ liệu MNIST với tối đa 50 kỷ nguyên, sử dụng callback ReduceLROnPlateau, EarlyStopping (restore_best_weights=True) và ModelCheckpoint (lưu best_qcnn.h5). Kết quả đánh giá trên 10.000 ảnh kiểm tra: MSE = 0,018665 (cải thiện 37,8% so với QAE Dense ~0,030) và PSNR = 17,29 dB (tăng 2,11 dB so với ~15,18 dB). Các chỉ số này xác nhận mô hình QCNN ConvAE có chất lượng tái tạo ảnh vượt trội về mặt phần mềm, song đòi hỏi tài nguyên tính toán lớn hơn đáng kể khi triển khai trên phần cứng.

4.5.3 Phân tích Tình trạng Tràn Tổng hợp FPGA - PRE-SYNTHESIS FAILED

Khi tiến hành chuyển đổi mô hình QCNN ConvAE sang C++ thông qua file hls4ml_conversion.ipynb và tổng hợp logic trên Vitis HLS 2022.2 với cấu hình

io_stream, ap_fixed<16,6>, Strategy=Resource, nhằm mục tiêu chip xc7a100tcs324-1 (Artix-7 100T), quá trình C-Simulation thành công với 0 lỗi, xác nhận tính đúng đắn về chức năng. Tuy nhiên, tổng hợp C-to-RTL (csynth) thất bại với thông báo PRE-SYNTHESIS FAILED do ba nguyên nhân kỹ thuật cốt lõi.

Nguyên nhân gốc rễ là kiến trúc QCNN ConvAE 28×28 vượt quá giới hạn tài nguyên vật lý của chip Artix-7 100T. Cụ thể: biến trung gian layer25_out ($392 \text{ nơ-ron} \times 16\text{-bit} = 6.272\text{-bit}$) và layer11_out (8.232-bit) vượt giới hạn bus 4.096-bit của Vitis HLS [ERROR HLS 214-256]; lớp dec_conv2 ($28 \times 28 \times 16$ kênh) đòi hỏi đồng thời 16 line_buffer BRAM vượt khả năng cấp phát của chip; và lớp UpSampling2D chưa được hls4ml hỗ trợ đầy đủ ở chế độ io_stream. Các lỗi này không phải giới hạn của công cụ tổng hợp mà phản ánh thực tế: với dòng FPGA phân khúc thấp như Artix-7 (135 BRAM, 240 DSP), kiến trúc CNN xử lý ảnh toàn kích thước 28×28 với decoder UpSampling2D vượt quá khả năng phân cứng hiện có.

Kết quả này cung cấp bằng chứng thực tiễn quan trọng: đối với các dòng FPGA có ràng buộc tài nguyên nghiêm ngặt, việc ưu tiên các kiến trúc mô hình tinh gọn là điều tất yếu. Điều này củng cố tính đúng đắn của lựa chọn kiến trúc QAE Dense ($196 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 196$) làm mô hình triển khai chính trong đề án.

4.5.4 Tổng hợp So sánh QAE Dense và QCNN ConvAE

Bảng 4.6 dưới đây tổng hợp đầy đủ sự đối chiếu giữa hai kiến trúc mạng Tự mã hóa lượng tử hóa trong đề án, từ thông số phần mềm đến kết quả triển khai phần cứng FPGA.

Bảng 4.6 So sánh toàn diện QAE Dense (mô hình gốc) và QCNN ConvAE (mô hình mở rộng)

Tiêu chí	QAE Dense (Mô hình gốc)	QCNN ConvAE (Mô hình mở rộng)
Kiến trúc mạng	$196 \rightarrow 16 \rightarrow 8 \rightarrow 16 \rightarrow 196$ (Dense thuần túy)	QConv2D+MaxPool+QDense +UpSampling2D+QConv2D

Kích thước đầu vào	196 chiều (14×14 flatten)	784 chiều (28×28×1 ảnh đầy đủ)
Không gian ẩn	8 chiều	16 chiều
Tổng tham số	6.764	16.169
Số phép MAC	6.528	1.480.192 (~227×)
Lượng tử hóa	8-bit QKeras	8-bit QKeras
MSE (test set)	~0,030	0,018665 (cải thiện 37,8%)
PSNR (test set)	~15,18 dB	17,29 dB (+2,11 dB)
C-Simulation	PASS	PASS
Tổng hợp FPGA	PASS - 91 chu kỳ	FAIL - PRE-SYNTHESIS
Độ trễ	0,91 μ s	N/A
Thông lượng	1,56 M FPS	N/A
Công suất tiêu thụ	0,665 W	N/A
Năng lượng/ảnh	0,604 μ J	N/A
DSP sử dụng	50% (120/240)	N/A
BRAM sử dụng	47,41% (64/135)	N/A
Kết luận FPGA	Triển khai thành công	Thất bại - vượt giới hạn

Từ kết quả thực nghiệm tổng hợp, mô hình QCNN ConvAE cho thấy chất lượng tái tạo ảnh vượt trội hơn trên môi trường phần mềm (MSE tốt hơn 37,8%, PSNR tăng 2,11 dB), song không thể được tổng hợp thành công trên phần cứng FPGA do vượt quá giới hạn tài nguyên và khả năng của chuỗi công cụ hls4ml/Vitis HLS. Điều này khẳng định nguyên lý thiết kế nhúng: hiệu năng thuật toán và khả năng triển khai phần cứng là hai tiêu chí độc lập. Mô hình QAE Dense với 6.528 phép MAC và kiến trúc Dense thuần túy là lựa chọn tối ưu cho chip Artix-7 100T trong ràng buộc tài nguyên của đồ án.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN NHỮNG ĐÓNG GÓP CHÍNH CỦA ĐỀ TÀI

Trong phạm vi chương trình đào tạo kỹ sư Điện - Điện tử, đề án đã hoàn thành việc thiết kế, tối ưu hóa và triển khai mô hình mạng tự mã hóa (Autoencoder) lên nền tảng phần cứng vi mạch khả trình. Việc đánh giá khách quan các kết quả thu được từ quá trình mô phỏng và tổng hợp cho thấy đề tài đã đáp ứng các mục tiêu kỹ thuật đặt ra, cụ thể trên ba phương diện chính như sau.

Thứ nhất, về khía cạnh xây dựng mô hình thuật toán, đề tài đã thiết kế cấu trúc mạng nơ-ron đối xứng với 5 lớp (196-16-8-16-196) nhằm trích xuất đặc trưng hình thái từ tập dữ liệu MNIST. Điểm trọng tâm của quá trình này là việc thực thi kỹ thuật lượng tử hóa 8-bit (8-bit Integer Quantization) cho toàn bộ ma trận trọng số và giá trị kích hoạt. Đánh giá định lượng cho thấy, phương pháp ép kiểu dữ liệu đã thu gọn đáng kể dung lượng lưu trữ của mô hình trong khi vẫn duy trì được năng lực phục dựng hình ảnh. Kết quả thực nghiệm ghi nhận chỉ số sai số toàn phương trung bình (MSE) trên tập dữ liệu kiểm tra đạt 0.030 và tỷ số tín hiệu trên nhiễu đỉnh (PSNR) đạt 15.18 dB.

Thứ hai, về khía cạnh kiến trúc và tối ưu hóa vi mạch, đề án đã hiện thực hóa thuật toán thành cấu trúc phần cứng thông qua luồng tổng hợp mức cao (HLS) trên môi trường phần mềm Vivado. Giải pháp ánh xạ tập trung các phép toán nhân cộng tích lũy (MAC) vào tài nguyên lõi cứng DSP48E1 thay vì sử dụng các phần tử logic phân tán đã giúp giảm thiểu diện tích tổng hợp. Đồng thời, việc can thiệp bằng chỉ thị tối ưu hóa định tuyến (Dataflow directive) đã thiết lập thành công cơ chế đường ống (pipeline) ở mức tác vụ. Cơ chế này cho phép các lớp nơ-ron hoạt động gối đầu lên nhau, giúp quản lý luồng dữ liệu vào/ra liên tục và hạn chế điểm nghẽn bộ nhớ cục bộ.

Thứ ba, về hiệu năng thực thi trên nền tảng phần cứng, báo cáo tổng hợp từ hệ thống cho thấy vi mạch mục tiêu Xilinx Artix-7 (xc7a100tcs324-1) đã đáp ứng các ràng buộc về định thời (meet timing) tại tần số xung nhịp 100 MHz, với chu kỳ thực tế đo lường đạt 7.180 ns. Các thông số về độ trễ (latency) được ghi nhận ở mức 91 chu kỳ xung nhịp (tương đương 0.910 μ s) cho một lượt suy luận hoàn

chính. Đồng thời, chỉ số thời gian khởi tạo (Initiation Interval - II) đạt 64 chu kỳ, đảm bảo duy trì thông lượng xử lý dữ liệu ổn định theo chu kỳ đồng hồ.

Tổng kết lại, các thông số thực nghiệm thu được đã chứng minh tính khả thi của quy trình thiết kế từ cấp độ mô hình phần mềm đến mức chuyển dạng thanh ghi (RTL). Đồ án cung cấp một nguyên mẫu phần cứng hoạt động ổn định kèm theo các số liệu đánh giá khách quan, làm cơ sở kỹ thuật cho việc ứng dụng và triển khai các mạng học sâu lên các thiết bị Edge AI bị giới hạn về tài nguyên vật lý trong tương lai.

5.2 NHỮNG MẶT HẠN CHẾ CÒN TỒN TẠI

Mặc dù hệ thống đã đáp ứng được các chỉ số kỹ thuật cơ bản trong quá trình tổng hợp và mô phỏng, đồ án vẫn còn một số giới hạn nhất định về phương pháp tiếp cận cũng như khả năng tích hợp thực tiễn. Việc phân tích khách quan những điểm hạn chế này là cơ sở quan trọng để đánh giá đúng không gian ứng dụng của đề tài.

Về mặt quy mô thuật toán và dữ liệu đầu vào, mô hình tự mã hóa hiện tại mới chỉ được huấn luyện và kiểm chứng tính khả thi trên tập dữ liệu ảnh chữ số viết tay MNIST với độ phân giải thấp (đã được nén xuống 14x14 điểm ảnh) và ở định dạng ảnh xám đơn kênh. Kiến trúc mạng với nút thất cổ chai 8 nơ-ron chưa được thử nghiệm khả năng trích xuất đặc trưng trên các bộ dữ liệu có độ phức tạp cao hơn, đa kênh màu (RGB) hoặc có nhiều nền ngẫu nhiên. Khi xử lý các luồng dữ liệu phức tạp, phương pháp ép kiểu dữ liệu xuống định dạng số nguyên 8-bit tĩnh (Static 8-bit Quantization) hiện tại có nguy cơ khuếch đại sai số nội suy, dẫn đến việc mất mát các đặc trưng hình thái quan trọng và làm suy giảm nghiêm trọng độ phân giải của ảnh phục dựng.

Bên cạnh yếu tố phần mềm, phương pháp luận thiết kế phần cứng được sử dụng trong đồ án cũng mang theo những giới hạn cố hữu. Đề tài lựa chọn luồng tổng hợp mức cao (HLS) làm công cụ biên dịch chính để ánh xạ mã C/C++ thành mạch kỹ thuật số. Mặc dù Vivado HLS cho phép rút ngắn đáng kể chu kỳ phát triển sản phẩm và hỗ trợ tạo luồng đường ống (pipeline) tự động, song phương pháp này vẫn tồn tại nhược điểm về tối ưu hóa diện tích. So với luồng thiết kế truyền thống bằng cách lập trình thủ công ở mức thanh ghi RTL (sử dụng Verilog

hoặc VHDL), mã RTL do HLS sinh ra thường tiêu tốn nhiều tài nguyên bảng tra cứu (LUTs) và bộ nhớ cục bộ hơn, đồng thời khó đạt được mức tần số xung nhịp tối đa (Fmax) tối ưu nhất do trình biên dịch tự động thường tạo ra các đường dẫn găng (critical paths) dài hơn so với việc tối ưu hóa thủ công.

Mặt khác, xét trên phương diện tích hợp hệ thống đo lường thực tế, cấu trúc vi mạch hiện tại mới dừng lại ở việc đóng gói thành một lõi sở hữu trí tuệ (IP Core) độc lập phục vụ chức năng gia tốc tính toán. Hệ thống chưa được phát triển thành một kiến trúc vi xử lý lai (SoC) hoàn chỉnh. Do đó, quy trình kiểm chứng phần cứng vẫn phụ thuộc vào luồng dữ liệu giả lập từ tệp testbench (Co-simulation) hoặc lưu trữ tĩnh trong bộ nhớ của khối FPGA. Đề tài chưa xây dựng được cơ sở hạ tầng giao tiếp ngoại vi, điển hình như bộ điều khiển truy cập bộ nhớ trực tiếp (AXI DMA), để truyền phát luồng hình ảnh theo thời gian thực từ cảm biến camera hoặc máy tính chủ xuống vi mạch nhằm đánh giá toàn diện độ trễ ở cấp độ hệ thống.

5.3 HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

Dựa trên những giới hạn đã được phân tích và đánh giá, định hướng phát triển của đề tài trong giai đoạn tiếp theo sẽ tập trung vào việc nâng cấp toàn diện cả về chiều sâu thuật toán lẫn khả năng mở rộng tích hợp ở cấp độ hệ thống phần cứng. Những cải tiến này nhằm mục đích biến nguyên mẫu vi mạch hiện tại thành một giải pháp Edge AI hoàn chỉnh và linh hoạt hơn.

Về khía cạnh nâng cấp thuật toán học sâu, hệ thống cần được mở rộng khả năng xử lý để tương thích với các bộ dữ liệu ảnh đa kênh màu (RGB) và có độ phân giải cao hơn, điển hình như CIFAR-10 hoặc ảnh thu nhận trực tiếp từ môi trường thực tế. Để làm được điều này, kiến trúc mạng tự mã hóa tuyến tính hiện tại có thể được nâng cấp lên mạng nơ-ron tích chập (Convolutional Autoencoder) nhằm tối ưu hóa khả năng trích xuất các đặc trưng không gian đa chiều. Song song đó, thay vì áp dụng lượng tử hóa tĩnh 8-bit toàn cục, thiết kế có thể nghiên cứu ứng dụng các kỹ thuật lượng tử hóa nhận biết huấn luyện (QAT) hoặc lượng tử hóa độ chính xác hỗn hợp (Mixed-precision Quantization). Giải pháp này sẽ cấp phát linh hoạt các định dạng bit khác nhau (ví dụ: 4-bit cho lớp ẩn, 8-bit cho lớp đầu ra) nhằm tìm ra điểm cân bằng tối ưu nhất giữa dung lượng bộ nhớ phần cứng và chất lượng tín hiệu khôi phục.

Tiếp nối quá trình hoàn thiện phần cứng, mục tiêu kỹ thuật quan trọng tiếp theo là chuyển đổi lõi IP độc lập hiện tại thành một hệ thống vi xử lý lai (SoC) toàn diện trên nền tảng FPGA. Hướng phát triển này đòi hỏi việc nhúng một bộ vi xử lý mềm (như lõi MicroBlaze) lên chip Artix-7 để làm nhiệm vụ điều phối trung tâm. Đồng thời, thiết kế cần xây dựng cơ sở hạ tầng giao tiếp chuẩn AXI4, đặc biệt là tích hợp luồng truy cập bộ nhớ trực tiếp (AXI DMA) và chuẩn giao tiếp AXI4-Stream. Cơ sở hạ tầng này sẽ tạo ra một đường ống băng thông rộng, cho phép hệ thống thu nhận trực tiếp luồng video thời gian thực từ các cảm biến camera ngoại vi, đưa vào bộ gia tốc giải mã và xuất tín hiệu ra màn hình hiển thị mà không bị thất cổ chai ở khâu truyền tải dữ liệu.

Mở rộng tầm nhìn về hệ sinh thái vạn vật kết nối (IoT), bộ gia tốc vi mạch khả trình này có tiềm năng được phát triển thành một nút điện toán biên (Edge Node) thông minh. Dữ liệu sau khi được phân tích và nội suy bởi FPGA có thể được gửi đến các máy tính nhúng hoặc vi điều khiển trung tâm để xử lý hậu kỳ. Thông qua việc tích hợp các giao thức truyền tải bản tin nhẹ, độ trễ thấp như MQTT, hệ thống có khả năng tương tác liên tục với máy chủ đám mây hoặc điều khiển chéo các thiết bị ngoại vi khác trong cùng mạng lưới. Sự kết hợp giữa năng lực xử lý số liệu mạnh mẽ của FPGA và tính linh hoạt của các giao thức truyền thông IoT sẽ mở ra triển vọng ứng dụng to lớn trong các lĩnh vực như hệ thống giám sát y tế thông minh, nhận diện cử chỉ, hoặc mạng lưới cảnh báo tự động trong công nghiệp.

TÀI LIỆU THAM KHẢO

- [1] M. Capra, B. Bussolino, A. Marchisio, M. Shafique, G. Masera, and M. Martina, “An updated survey of efficient hardware architectures for accelerating deep convolutional neural networks,” *Future Internet*, vol. 12, no. 7, p. 113, 2020.
- [2] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O’Reilly Media, 2019.
- [3] M. Isik et al., “An efficient hardware architecture for autoencoder neural networks.”
- [4] Ian Goodfellow, Yoshua Bengio, Aaron Courville, “Deep Learning”, MIT Press, 2016.
- [5] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [6] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.
- [7] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv:1308.3432*, 2013.
- [8] C. N. Coelho Jr. et al., “Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors,” *Nature Machine Intelligence*, vol. 3, no. 8, pp. 675–686, 2021.
- [9] Xilinx, “7 Series FPGAs Configurable Logic Block User Guide (UG474),” 2016.
- [10] Xilinx, “7 Series DSP48E1 Slice User Guide (UG479) , Xilinx Inc., 2018.”
- [11] Xilinx, “Vitis HLS User Guide (UG1399),” 2022.

- [12] F. Fahim et al., “hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices,” arXiv:2103.05579, 2021.
- [13] J. Duarte et al., “Fast inference of deep neural networks in FPGAs for particle physics,” Journal of Instrumentation, vol. 13, P07027, 2018.
- [14] Xilinx, “AXI Reference Guide (UG761), Xilinx Inc., 2012.”